# STRUCTURAL AUGMENTED REASONING FOR LANGUAGE MODELS

by

**YUBO WANG**

A Thesis Submitted to
The Hong Kong University of Science and Technology
in Partial Fulfillment of the Requirements for
the Degree of Doctor of Philosophy
in Computer Science and Engineering

May 2026, Hong Kong

Copyright © by Yubo Wang 2026

# Authorization

I hereby declare that I am the sole author of the thesis.

I authorize the Hong Kong University of Science and Technology to lend this thesis to other institutions or individuals for the purpose of scholarly research.

I further authorize the Hong Kong University of Science and Technology to reproduce the thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

_____

YUBO WANG

# STRUCTURAL AUGMENTED REASONING FOR LANGUAGE MODELS

by

**YUBO WANG**

This is to certify that I have examined the above Ph.D. thesis
and have found that it is complete and satisfactory in all respects,
and that any and all revisions required by
the thesis examination committee have been made.

---

Prof. Lei Chen, Thesis Supervisor

---

Prof. Xiaofang Zhou, Head of Department

Department of Computer Science and Engineering

22 May 2026

# ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Lei Chen, for his unwavering guidance, insightful advice, and continuous support throughout my doctoral studies. His rigorous attitude toward research, broad academic vision, and patient mentorship have profoundly shaped the way I formulate problems, conduct experiments, and communicate ideas. The lessons I have learned from him will continue to benefit me far beyond the completion of this thesis.

I would also like to extend my sincere thanks to the members of my thesis proposal committee, Professor Dan Xu, who kindly serves as the chairperson, and Professor Chaojian Li. Their thoughtful comments, constructive criticism, and valuable suggestions have substantially improved the quality and clarity of this work, and have inspired several promising directions for future research.

I am grateful to the Department of Computer Science and Engineering at the Hong Kong University of Science and Technology for providing an outstanding academic environment, generous resources, and a vibrant research community that has made this journey both challenging and rewarding.

I sincerely thank my labmates and fellow students for the countless discussions, collaborations, and shared moments of joy and difficulty. Their companionship and intellectual exchange have made my graduate study a truly memorable experience.

Last but not least, I am deeply indebted to my family for their unconditional love, patience, and encouragement. Their constant support has been my greatest source of strength throughout this journey.

# TABLE OF CONTENTS

**Title Page** i

**Authorization Page** ii

**Signature Page** iii

**Acknowledgments** iv

**Table of Contents** v

**List of Figures** ix

**List of Tables** x

**Abstract** xi

**Chapter 1 Introduction** 1

1.1 Research Motivation 1

1.2 Problem Statement 3

1.2.1 Problem 1: Knowledge-Intensive Tabular Understanding 3

1.2.2 Problem 2: Dynamic Few-Shot Text Classification 4

1.2.3 Problem 3: General-Purpose Graph-Augmented RAG 5

1.3 Overview of Contributions 6

1.4 Thesis Organization 8

**Chapter 2 Related Work** 10

2.1 Knowledge Graphs for Data Understanding 10

2.1.1 Knowledge Graphs and Their Applications 10

2.1.2 Knowledge-Graph-Based Column and Table Annotation 11

2.1.3 Deep Learning Approaches to Table Understanding 12

2.1.4 Limitations of Existing Approaches 12

2.2 Retrieval-Augmented Generation 13

2.2.1 The RAG Paradigm 13
2.2.2 Dense Passage Retrieval 13
2.2.3 Graph-Based Retrieval-Augmented Generation 14
2.2.4 Comparison: Vector Retrieval versus Graph-Structured Retrieval 15
2.2.5 Current Limitations and Open Challenges 15
2.3 Few-Shot Text Classification 16
2.3.1 Traditional Text Classification Methods 16
2.3.2 Few-Shot Learning Paradigms 16
2.3.3 LLM-Based Few-Shot Classification 17
2.3.4 Dynamic Label Settings and Online Learning 17
2.3.5 Social Media Text Classification Applications 18
2.4 Graph Algorithms for Information Retrieval 18
2.4.1 Subgraph Matching 18
2.4.2 Personalized PageRank and Influence Propagation 19
2.4.3 Minimum Spanning Tree and Its Variants 19
2.4.4 Graph-Based Reasoning and Inference 20
2.5 Summary and Research Gaps 20
2.5.1 Comparative Summary of Existing Approaches 20
2.5.2 Specific Research Gaps Addressed by This Thesis 21

**Chapter 3 KGLink: Knowledge Graph Enhanced Column Type Annotation 23**
3.1 Introduction 23
3.2 Problem Formulation 24
3.2.1 Notation and Task Definition 24
3.2.2 An Illustrative Example 25
3.3 The KGLink Framework 26
3.3.1 Overview 26
3.3.2 Knowledge Graph Linking Module 27
3.3.3 Pre-trained Language Model Encoding Module 29
3.3.4 Hybrid Reasoning and Type Prediction 30
3.4 Addressing Type Granularity 30
3.4.1 Why Granularity Is Hard for KG-only Methods 31
3.4.2 KGLink's Solution: Hierarchy-Constrained Selection 31

3.4.3 Theoretical Justification 32
3.5 Addressing Missing Context 32
3.5.1 When Does Context Go Missing? 32
3.5.2 Knowledge Graphs as External Context 33
3.5.3 Integration Mechanism 33
3.6 Experiments 34
3.6.1 Experimental Setup 34
3.6.2 Main Results 35
3.6.3 Ablation Study 37
3.6.4 Granularity Analysis 38
3.6.5 Context Analysis 38
3.7 Summary 38

**Chapter 4 GORAG: Graph-based Online Retrieval Augmented Generation 40**
4.1 Introduction 40
4.2 Problem Formulation 43
4.2.1 Notation and Preliminaries 43
4.2.2 Dynamic Few-Shot Text Classification 43
4.2.3 Retrieval-Augmented Generation Formulation 44
4.2.4 Limitations of Existing Formulations 44
4.3 Analysis of Existing Graph-RAG Limitations 45
4.3.1 The Uniform Indexing Problem 45
4.3.2 The Non-Adaptive Retrieval Problem 46
4.3.3 The Incomplete Context Problem 46
4.4 The GORAG Framework 48
4.4.1 Overview 48
4.4.2 Weighted Graph Construction 48
4.4.3 Online Graph Maintenance 50
4.4.4 Adaptive Retrieval via Minimum-Cost Spanning Tree 51
4.4.5 Context Assembly and LLM Prompting 54
4.5 Experiments 55
4.5.1 Experimental Setup 55
4.5.2 Main Results 56

4.5.3 Ablation Study 57
4.5.4 Analysis 58
4.6 Summary 59

**Chapter 5 Proposed Future Work 61**
5.1 Research Direction Overview 61
5.2 Advanced Graph-based Retrieval-Augmented Generation 63
5.2.1 Motivation and Problem Definition 63
5.2.2 Proposed Approach 64
5.2.3 Planned Experiments 67
5.2.4 Expected Contributions 68
5.3 Semantic Subgraph Matching and Graph-Aware Caching 68
5.3.1 Motivation and Problem Definition 68
5.3.2 Proposed Approach 69
5.3.3 Planned Experiments 70
5.3.4 Expected Contributions 72
5.4 Research Timeline 72

**Chapter 6 Conclusion and Expected Contributions 74**
6.1 Summary of Completed Work 74
6.2 Expected Contributions 75
6.2.1 Theoretical Contributions 75
6.2.2 Methodological Contributions 76
6.2.3 Practical Contributions 77
6.3 Potential Risks and Mitigation Strategies 77
6.3.1 Technical Risks 77
6.3.2 Data and Evaluation Risks 78
6.3.3 Timeline Risks 79
6.4 Alternative Plans 80
6.5 Concluding Remarks 81

**References 82**

# LIST OF FIGURES

3.2 High-level architecture of KGLink. The KG linking module produces a candidate type distribution from external knowledge, while the PLM encoding module captures contextual signals from the table itself; a hybrid reasoning module fuses the two sources to produce the final prediction. 26

4.1 An overview of GORAG in each round. In Part 1, GORAG constructs a weighted graph based on keywords extracted from the few-shot training data. In Part 2, GORAG performs adaptive graph retrieval and outputs the candidate labels. In Part 3, GORAG classifies texts into one of the candidate labels and applies online graph indexing to update the graph. 47

4.2 Skeleton of the GORAG prompt template (full templates per task are released with the code). 54

5.1 Overview of the proposed research programme and its relationship to the completed works. 62

5.2 Proposed three-step framework for advanced graph-based RAG. 65

5.3 Proposed framework for semantic subgraph matching and graph-aware KV caching. 71

# LIST OF TABLES


2.1 Comparison of existing approaches along five capability axes 21

3.1 Statistics of the two CTA benchmarks used in our experiments. 34

3.2 Column type annotation results on SemTab and VizNet 36

3.3 Performance breakdown by column type on SemTab 37

4.1 Dataset statistics for GORAG experiments 55

4.2 Main results: macro-F1 (↑) on three datasets at $K \in \{1, 5, 10\}$. Best in bold, runner-up underlined. 57

4.3 Ablation on Misinformation at $K = 5$. 58

5.1 Planned research timeline (2026–2028). 73

# STRUCTURAL AUGMENTED REASONING FOR LANGUAGE MODELS

**by**

**YUBO WANG**

Department of Computer Science and Engineering

The Hong Kong University of Science and Technology

## ABSTRACT

Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation. However, their reasoning processes often rely on shallow pattern matching over unstructured textual inputs, which limits their effectiveness in scenarios that demand external domain knowledge, fine-grained entity-level understanding, or rapid adaptation to evolving tasks under minimal supervision. These limitations become particularly pronounced in knowledge-intensive applications, dynamic real-world environments, and downstream tasks where labeled data is scarce.

This thesis proposes *structural augmented reasoning*, a unified paradigm that integrates explicit structured representations, including knowledge graphs and dynamically constructed weighted graphs, into language model pipelines to enable more precise, context-aware, and adaptive reasoning. The thesis develops this paradigm through two completed studies and one ongoing investigation.

In the first completed work, KGLink, we present a hybrid framework that couples knowledge graph information with pre-trained language models for knowledge-intensive tabular understanding. KGLink explicitly addresses the type granularity mismatch and

missing context issues that hinder existing column type annotation methods, yielding substantial gains in both accuracy and robustness. In the second completed work, GORAG, we propose a graph-based online retrieval-augmented generation framework for dynamic few-shot social media text classification. GORAG constructs and continually maintains an adaptive weighted graph that supports threshold-free, input-specific retrieval through minimum-cost spanning trees, and has been published at WWW 2026.

Building upon these foundations, the proposed work extends structural augmented reasoning to general-purpose retrieval-augmented generation systems through two complementary directions: (1) advanced graph-based retrieval that produces explicit reasoning chains and influence-aware subgraphs, and (2) path-aware semantic subgraph matching coupled with graph-aware caching mechanisms for improved retrieval efficiency. The expected contributions span both methodological advances and practical impact across knowledge-intensive applications.

# CHAPTER 1

# INTRODUCTION

## 1.1 Research Motivation

Large language models (LLMs) such as BERT [10], GPT-3 [4], and GPT-4 [26], all built upon the Transformer architecture [36], have produced striking advances on a wide range of natural language understanding (NLU) and natural language generation (NLG) benchmarks. By pre-training on internet-scale corpora and fine-tuning or prompting on downstream data, these models acquire surprisingly broad linguistic competence and surface-level world knowledge. Despite these successes, a growing body of evidence indicates that the apparent "reasoning" performed by LLMs is often a form of shallow pattern matching over distributional statistics rather than principled multi-step inference grounded in explicit knowledge structures. This gap between surface fluency and true reasoning has become increasingly visible as LLMs are deployed in knowledge-intensive applications such as scientific question answering, biomedical decision support, financial analysis, and content moderation, where the cost of plausible-sounding but factually incorrect outputs is high.

Three concrete limitations crystallize this gap. First, LLMs lack reliable access to *external domain knowledge*. The parametric memory of an LLM is bounded by its training corpus, is expensive to update, and frequently leads to hallucinations [13] when the model is queried about long-tail entities, recent events, or specialized domains. Knowledge graphs (KGs) such as Wikidata [38] and DBpedia [2] encode structured, curated knowledge about millions of entities and relations, yet integrating this structured knowledge with the unstructured representations learned by LLMs remains a fundamentally open problem [27, 15]. Second, LLMs exhibit insufficient *entity-level understanding*: they tend to reason at the level of tokens or phrases rather than entities, relations, and semantic types. As a consequence, tasks that require precise grounding to a specific entity in a knowledge base—such as column type annotation, entity linking, or table understanding—continue

to challenge state-of-the-art LLMs even when their language modeling perplexity is low. Third, LLMs adapt poorly to *evolving tasks with minimal supervision*. In real-world deployments, the label space is rarely fixed: new categories of misinformation, new product taxonomies, and new types of scientific entities emerge continuously, and the per-class supervision available is typically only a handful of examples. Standard fine-tuning is impractical at this scale, and few-shot in-context prompting [4] is sensitive to demonstration selection and provides no principled mechanism for retrieving the right context.

A unifying response to these three limitations is to equip language models with *explicit structured representations*. Knowledge graphs provide a natural substrate for the first limitation by exposing curated entities, types, and relations to the language model. Heterogeneous graphs constructed from textual corpora—linking entities, keywords, exemplars, and labels—address the second limitation by giving the model a principled topology over which to reason at the entity level. Dynamically constructed and re-weighted graphs that adapt to each input address the third limitation by enabling the system to retrieve evidence in a manner that is sensitive to the specific structure of the query and the current label space. The overarching thesis of this proposal is that integrating such structured representations into language model pipelines—rather than treating LLMs as monolithic black boxes—yields substantially more precise, context-aware, and adaptive reasoning.

The retrieval-augmented generation (RAG) paradigm [24, 13] has emerged as the dominant practical strategy for grounding language models in external evidence. In its standard form, RAG retrieves text passages by dense vector similarity [22] and concatenates the top-ranked passages with the input prompt. While effective for many open-domain question answering settings, flat vector retrieval suffers from several well-documented weaknesses: it treats each passage as an independent unit and therefore loses the relational structure that connects entities across passages; it is governed by a fixed similarity threshold or top-$k$ cutoff that is poorly suited to queries of varying difficulty; and it provides no mechanism to express multi-hop reasoning chains over evidence. Recent work on graph-based RAG, including GraphRAG [11] and LightRAG [14], partially addresses these issues by organizing retrieved evidence into entity-relation graphs, but the resulting systems still rely on LLM-generated graph constructions that are prone to hallucination, employ ad hoc retrieval policies that ignore graph structure, and lack principled algorith-

mic foundations for selecting reasoning subgraphs.

This proposal argues that the next generation of language model reasoning systems should be built around *structural augmented reasoning*: a paradigm in which language models are systematically coupled with explicit graph structures—both pre-existing knowledge graphs and dynamically constructed weighted graphs—through principled retrieval, matching, and inference algorithms. Under this paradigm, the language model is no longer the sole repository of knowledge or the sole arbiter of relevance; instead, structured representations and graph algorithms shoulder the burden of organizing evidence, exposing reasoning paths, and adapting to evolving inputs, while the language model focuses on what it does best: interpreting natural language context and generating fluent, semantically appropriate output. The remainder of this chapter formalizes the specific research problems this proposal addresses, summarizes the contributions of the four works that constitute the thesis, and outlines the organization of the document.

## 1.2 Problem Statement

The unifying motivation outlined in Section 1.1 translates into three concrete research problems, each centered on a different combination of structured representation and language model. The first two have been addressed by published work; the third is the subject of proposed future research that builds directly on the first two.

### 1.2.1 Problem 1: Knowledge-Intensive Tabular Understanding

Tabular data underpins enterprise databases, web tables, and scientific datasets, and understanding the semantic type of each column—a task known as column type annotation (CTA)—is foundational for table search, data integration, and downstream analytics. Two broad families of methods have been pursued for CTA. *Knowledge-graph-based methods* match cell values against entities in a KG such as Wikidata or DBpedia and aggregate the types of matched entities to predict a column type. These methods inherit the rich semantic granularity of the underlying KG but suffer from two persistent issues. First, the type granularity exposed by a KG is rarely a perfect match for the granularity required

by a downstream task: a KG may predict the fine-grained type `Q5` (human) for a column whose target label in a benchmark is the coarser type `Person`, or vice versa, leading to systematic misalignment. Second, the scalability of KG-based matching deteriorates rapidly when the value vocabulary is large, when entity surface forms are noisy, or when the column contains entities that are missing from the KG.

*Deep-learning-based methods* treat CTA as a text classification problem over column values, often serializing the column (and sometimes its surrounding table context) and feeding it through a pre-trained language model [10]. These approaches are flexible with respect to the target type vocabulary and handle noisy text gracefully, but they consume only the information explicitly present in the table. When the table provides insufficient context—for instance, a column of bare numeric codes whose meaning is documented elsewhere—deep-learning-based CTA models silently fall back on shallow lexical heuristics. Neither family of methods alone satisfies the requirements of practical CTA: KG methods are precise where the KG covers the data but brittle where it does not, and deep-learning methods are robust to noise but blind to external knowledge.

The first research problem of this thesis is therefore to design a *hybrid framework* that combines the broad coverage and semantic precision of KG-based matching with the contextual flexibility of pre-trained language models, while explicitly aligning type granularity to the target task. The framework must (i) recover column-relevant evidence from a large-scale KG efficiently, (ii) reconcile mismatches between KG type granularity and target type vocabularies, and (iii) compensate for cases in which the table itself provides insufficient signal. This problem is addressed by KGLink [40], summarized in Section 1.3 and detailed in later chapters.

### 1.2.2 Problem 2: Dynamic Few-Shot Text Classification

Many real-world text classification settings are characterized by an evolving label space and a chronic scarcity of labeled data. Social media moderation is the canonical example: the categories of hate speech, misinformation, scam content, and platform policy violations shift on time scales of weeks or months, and the labeling effort required to keep pace with these shifts is prohibitive. Even when each new label is provided with only a handful of seed examples, downstream classifiers must generalize from those examples to a stream

of incoming user-generated content that is noisy, abbreviated, and full of platform-specific idiom.

Standard few-shot learning methods such as prototypical networks [32] assume a fixed embedding space and a relatively static class structure, and therefore degrade when the label set evolves over time. LLM-based few-shot classification via in-context learning is more flexible but is sensitive to demonstration selection, sees only a small fixed window of context, and lacks any principled mechanism for retrieving the most informative seed examples. Graph-based RAG approaches [11, 14] could in principle address this gap by indexing the seed examples as a graph and retrieving structurally relevant subgraphs at inference time. However, current graph-based RAG systems exhibit three failure modes that make them ill-suited to dynamic few-shot classification. First, they construct a *uniform index* that treats all entities and edges with equal weight, ignoring the fact that some keywords are far more diagnostic of a target label than others. Second, their retrieval policies are *non-adaptive*: they apply the same top-k or threshold rule to every query, even though some queries demand only a single highly relevant exemplar while others require a constellation of weakly related cues. Third, the retrieved context is often *incomplete* along the most informative reasoning path, because pruning is performed by similarity rather than by structural connectedness through the graph.

The second research problem of this thesis is therefore to design an *adaptive graph-based retrieval framework* for dynamic few-shot text classification that (i) constructs a heterogeneous graph linking labels, keywords, and exemplars with edge weights that reflect the discriminative power of each connection; (ii) supports online updates as new labels and new examples arrive without re-indexing; and (iii) retrieves a query-specific reasoning subgraph whose extent is determined by the structure of the graph itself rather than by an arbitrary cutoff. This problem is addressed by GORAG [39], also summarized below.

### 1.2.3 Problem 3: General-Purpose Graph-Augmented RAG

The first two problems are tied to specific application domains—tabular understanding and dynamic text classification—but the underlying methodological gap is far more general. As graph-augmented RAG matures into a default architectural pattern for knowledge-grounded language models, three structural weaknesses become increasingly visible across

applications. First, the graphs themselves are typically constructed by prompting an LLM to extract entities and relations from raw text, and *LLM hallucination during graph construction* introduces phantom nodes and spurious edges that propagate downstream. Second, the reasoning chains exposed by current systems are *incomplete*: retrieval returns a set of nodes or a single shortest path, but rarely a coherent subgraph that supports multi-hop inference. Third, current systems perform *inflexible graph matching* by relying on either rigid exact-subgraph isomorphism in the style of VF2 [7] or purely embedding-based similarity, neither of which captures the semantic flexibility required when the user query and the indexed graph are described in different vocabularies.

The third research problem of this thesis, posed as proposed future work, is to design a *general-purpose graph-augmented RAG framework* that (i) extracts entities and constructs the underlying graph using lightweight statistical signals such as TF-IDF rather than relying solely on LLM prompting, thereby reducing hallucination at the source; (ii) selects an evidence subgraph by jointly optimizing relevance and connectivity through a minimum-cost maximum-influence (MCMI) criterion, exposing explicit reasoning chains rather than disconnected node sets; and (iii) supports semantic subgraph matching with graph-aware caching, so that path-level reasoning over indexed graphs can be performed efficiently at inference time.

## 1.3 Overview of Contributions

This thesis contributes four interconnected pieces of work that together instantiate the structural augmented reasoning paradigm. The first two have been published or accepted; the latter two are the subject of proposed research and are described in this proposal as future contributions.

**Contribution 1: KGLink—hybrid KG and PLM for column type annotation.** KGLink [40] is a framework for column type annotation that couples a knowledge graph with a pre-trained language model. KGLink performs entity matching against a KG to recover candidate types for each cell value, but rather than relying on the raw KG type as a label, it constructs a granularity-aware mapping between KG types and the target type vocab-

ulary, resolving the granularity mismatch problem that limits pure KG-based methods. In parallel, KGLink encodes the column values together with auxiliary KG-derived context using a pre-trained language model, allowing the system to compensate for tables in which the explicit cell content is insufficient. The two streams of evidence are fused through a learned aggregation mechanism that adapts to the per-column reliability of each source. KGLink achieves substantial improvements over both pure KG-based and pure deep-learning-based baselines on standard CTA benchmarks and is the first framework, to the best of our knowledge, to systematically address granularity and missing context in a single architecture.

**Contribution 2: GORAG—graph-based online RAG for dynamic few-shot classification.** GORAG [39] addresses the dynamic few-shot text classification problem by introducing an adaptive weighted graph that organizes labels, discriminative keywords, and exemplar texts into a heterogeneous structure with information-theoretic edge weights. GORAG supports online insertion of new labels and new exemplars without re-indexing. At inference time, it retrieves a query-specific subgraph through a minimum-cost spanning-tree formulation that connects the input text to the most informative labeled exemplars without relying on a fixed similarity threshold or a fixed top-k cutoff. GORAG demonstrates that adaptive, structure-aware retrieval significantly outperforms both flat dense retrieval and prior graph-based RAG baselines on social media classification benchmarks, particularly in settings where the label set evolves over time.

**Contribution 3 (Proposed): Advanced graph-based RAG with influence-aware subgraph generation.** Building on the lessons learned from GORAG, the third proposed contribution is a general-purpose graph-augmented RAG framework that targets the structural weaknesses of current graph-based RAG systems. The proposed framework constructs the underlying graph using a TF-IDF-based entity extraction pipeline, which reduces hallucination relative to fully LLM-driven extraction. For retrieval, the framework formulates evidence selection as a minimum-cost maximum-influence (MCMI) subgraph generation problem and develops a greedy algorithm with bounded approximation guarantees, drawing on prior work on personalized PageRank [25, 3] and influence propagation. The output of MCMI retrieval is a connected subgraph that exposes explicit, multi-

hop reasoning chains to the downstream language model, in contrast to the disconnected node sets returned by current systems.

**Contribution 4 (Proposed): Semantic subgraph matching with graph-aware caching.** The fourth proposed contribution focuses on efficient deployment of graph-augmented RAG at scale. Once the indexed graph and the per-query reasoning subgraph are available, an inherent bottleneck is the matching of query subgraphs against the indexed graph, particularly when matching must be semantic rather than purely structural. The proposed framework introduces *path-aware semantic subgraph matching*, which generalizes classical exact subgraph matching algorithms [7] to allow path-level semantic equivalence between query and index, and pairs this matching with a *graph-aware key–value cache* that exploits the structural overlap between successive queries to reuse intermediate retrieval and inference computations. The two components together aim to make principled graph-augmented reasoning practical at production-scale latency budgets.

Taken together, the four contributions form a coherent research arc. KGLink establishes that explicit KG structure can be reconciled with pre-trained language models in a single hybrid architecture for a knowledge-intensive task. GORAG shows that the same structural philosophy, when extended to dynamically constructed weighted graphs with adaptive retrieval, transfers to a fundamentally different setting (few-shot classification with an evolving label space). The two proposed contributions then generalize the lessons learned: first, by formulating principled graph algorithms for evidence selection in general-purpose RAG, and second, by addressing the systems-level question of how such graph-based reasoning can be matched and cached efficiently. Each contribution targets a distinct piece of the structural augmented reasoning paradigm, and the four together provide an end-to-end view of the design space.

## 1.4 Thesis Organization

The remainder of this proposal is organized as follows.

**Chapter 2** surveys the body of related work that situates this thesis. Section 2.1 reviews knowledge graphs and their use in tabular and entity-centric data understanding,

including a discussion of column type annotation methods. Section 2.2 surveys retrieval-augmented generation, contrasting dense passage retrieval with the emerging family of graph-based RAG approaches and identifying the structural limitations that motivate this thesis. Section 2.3 reviews few-shot text classification, paying particular attention to dynamic label settings and social-media applications. Section 2.4 covers the graph algorithms that underpin the proposed methods, including subgraph matching, personalized PageRank, and minimum spanning tree variants. Section 2.5 synthesizes the survey into an explicit gap analysis that motivates each of the four contributions.

**Chapter 3** presents KGLink, the hybrid knowledge-graph and pre-trained language-model framework for column type annotation. The chapter formalizes the column type annotation problem with attention to granularity mismatch, introduces the KGLink architecture in detail, and reports a comprehensive empirical evaluation against both KG-based and deep-learning-based baselines on standard benchmarks.

**Chapter 4** presents GORAG, the graph-based online RAG framework for dynamic few-shot text classification. The chapter introduces the dynamic few-shot setting, presents the adaptive weighted graph and the minimum-cost spanning-tree retrieval procedure, and reports experiments on social-media classification benchmarks under both static and evolving label conditions.

**Chapter 5** describes the proposed advanced graph-based RAG framework with TF-IDF-based entity extraction and minimum-cost maximum-influence subgraph generation. The chapter presents the formal problem definition, the greedy algorithm with its analysis, and a detailed plan for empirical evaluation against state-of-the-art graph-based RAG baselines.

**Chapter 5** also describes the proposed semantic subgraph matching framework with graph-aware key–value caching. The chapter outlines the path-aware matching formulation, the cache design, and the planned end-to-end evaluation in production-style RAG pipelines.

**Chapter 6** concludes the proposal with a summary of expected contributions, a discussion of broader implications for structural augmented reasoning, and a timeline for the proposed research.

# CHAPTER 2

# RELATED WORK

This chapter surveys the four bodies of literature that this thesis builds on and extends. Section 2.1 reviews knowledge graphs and their applications to data and tabular understanding, with a focus on column-level annotation. Section 2.2 reviews retrieval-augmented generation, including both dense passage retrieval and the emerging family of graph-based RAG methods. Section 2.3 surveys few-shot text classification methods relevant to the dynamic and online settings targeted in this thesis. Section 2.4 reviews the graph algorithms—subgraph matching, personalized PageRank, and minimum spanning tree variants—that provide the algorithmic foundations for the proposed work. Finally, Section 2.5 synthesizes these surveys into an explicit gap analysis, summarized in a comparison table, and identifies the four research gaps that this thesis addresses.

## 2.1 Knowledge Graphs for Data Understanding

### 2.1.1 Knowledge Graphs and Their Applications

A knowledge graph (KG) is a structured representation of factual knowledge in which real-world entities are modeled as nodes and the relations between entities are modeled as labeled edges. Large open-domain KGs such as Wikidata [38] and DBpedia [2] have grown to encompass tens of millions of entities and billions of triples, drawn from collaboratively curated and automatically extracted sources. Surveys of the field [15, 27] document a remarkable expansion in the construction, completion, and consumption of KGs over the last decade and a corresponding proliferation of downstream applications, ranging from semantic search to question answering, recommendation, biomedical informatics, and decision support.

The strengths of KGs as a substrate for data understanding are well established. They expose explicit, machine-interpretable types and relations, support multi-hop traversal, and provide a stable identifier space that resists the vocabulary drift typical of natural

language. These properties make KGs particularly attractive whenever a downstream task must reason at the level of entities rather than tokens, or whenever the system must justify its outputs through traceable evidence. Recent work [27] explicitly explores the unification of large language models and KGs, motivated precisely by the observation that the parametric memory of LLMs lacks both the precision and the auditability of explicit graphs.

Their weaknesses are equally well documented. Coverage is uneven: long-tail entities and recent events are systematically under-represented. Type granularity is non-uniform across domains, leading to schema mismatches when KG types must be aligned with task-specific label vocabularies. Entity surface forms are often ambiguous, and entity resolution between unstructured text and KG entries is itself a non-trivial problem. These weaknesses make pure KG-based approaches brittle in practice and motivate hybrid methods that combine KG structure with the flexibility of learned text representations.

### 2.1.2 Knowledge-Graph-Based Column and Table Annotation

Column type annotation (CTA) and the broader task of semantic table interpretation aim to assign each column of a table a semantic type drawn from an external vocabulary, often a KG ontology. Early approaches such as TableMiner [45] and SemTab-style systems [1] match cell values to KG entities, aggregate the types of matched entities, and resolve the column type by majority vote or probabilistic inference over a candidate set. These methods are precise when the KG covers the data well and when the granularity of KG types aligns with the target vocabulary, but they fail in two predictable ways. First, when the cell vocabulary contains entities not in the KG—abbreviations, compound names, or domain-specific identifiers—matching collapses and the column is left untyped. Second, when the KG type granularity is finer or coarser than the target labels, the aggregation step produces systematically biased predictions: a column annotated as `Q5` (human) in Wikidata may correspond to either a generic `Person` or a specific `Athlete` type in the target schema, and the KG alone cannot disambiguate.

A complementary thread of work explicitly grounds tabular cells through entity linking before performing annotation. Such methods improve precision by leveraging contextual signals from neighboring cells and column headers, but inherit the coverage lim-

itations of the underlying KG. Despite considerable algorithmic progress on candidate generation and disambiguation, no purely KG-based method has been shown to bridge the granularity gap in a principled way, motivating hybrid approaches that combine structured matching with learned representations.

### 2.1.3 Deep Learning Approaches to Table Understanding

A second family of CTA methods abandons explicit KGs and treats column annotation as a text classification problem over column values. Models such as Sherlock [17] and TURL [9] serialize columns—and sometimes their surrounding tables—and feed the resulting sequence through a pre-trained language model [10]. These models are trained to predict a column type from a fixed task-specific vocabulary, sidestepping granularity mismatch by learning the mapping from data to labels directly. They are robust to noisy text, surface-form variation, and out-of-KG entities, and they have set strong baselines on benchmarks where the table itself contains sufficient signal.

Their weakness is the converse of the KG-based approach: when the table provides insufficient context, deep-learning-based CTA models silently fall back on shallow lexical heuristics. A column of bare numeric codes, a column of acronyms, or a sparse column of free-text snippets provides little for the language model to encode, and external knowledge that would resolve the column unambiguously—present in a KG—is simply not available to the model. Recent table-pretraining systems such as TaBERT [43] and TUTA [41] introduce hierarchical table-aware encoders that improve over generic text encoders, but they do not address the missing-context problem in any fundamental way.

### 2.1.4 Limitations of Existing Approaches

The two families above expose complementary strengths and weaknesses. KG-based methods bring broad coverage and semantic precision but are brittle when the KG is incomplete or when type granularity is misaligned. Deep-learning-based methods bring flexibility and robustness but are blind to external knowledge and degrade silently on tables that lack explicit context. A small number of recent methods attempt to combine the two, but typically as loose feature concatenations rather than principled fusions, and very

few address granularity reconciliation explicitly. The gap motivates the KGLink contribution of this thesis: a hybrid framework in which KG-derived candidates and language-model-encoded context are reconciled through a granularity-aware aggregation mechanism that adapts to the per-column reliability of each evidence source.

## 2.2 Retrieval-Augmented Generation

### 2.2.1 The RAG Paradigm

Retrieval-augmented generation (RAG) [24, 13] couples a parametric language model with a non-parametric retriever over an external corpus. At inference time, the retriever fetches a small set of relevant documents conditioned on the input query, the documents are concatenated with the query, and the language model generates an output conditioned on the concatenation. The motivation is twofold: external corpora can be updated cheaply without retraining the language model, and the retriever supplies the model with evidence that may be absent or unreliable in its parametric memory. RAG has become a de facto standard for knowledge-intensive applications, and a growing literature explores its many components, from retriever architectures to prompt formats to end-to-end fine-tuning.

The basic RAG architecture has several attractive properties. It scales naturally to large external corpora, separates concerns between retrieval and generation, and provides at least a partial solution to the hallucination problem that plagues pure parametric models. At the same time, it inherits the limitations of the retriever it is built on, and the choice of retriever turns out to be the dominant factor in end-to-end RAG quality. The two architectural families that have received the most attention—dense passage retrieval and graph-based retrieval—differ fundamentally in how they organize and access evidence.

### 2.2.2 Dense Passage Retrieval

Dense passage retrieval (DPR) [22] encodes both queries and passages into a shared dense vector space using a pre-trained encoder, typically a dual-encoder architecture trained with contrastive objectives on (query, relevant-passage) pairs. Retrieval reduces to nearest-

neighbor search over the passage embeddings, with similarity computed by inner product or cosine. DPR significantly outperforms classical sparse retrievers such as BM25 on open-domain question answering benchmarks and has become the default starting point for RAG pipelines.

Despite its empirical success, DPR has structural limitations relevant to this thesis. First, it treats each passage as an isolated unit; relations between passages—shared entities, citations, conceptual overlap—are not represented in the index, and retrieval cannot exploit them. Second, the retrieval policy is fixed: a top-$k$ cutoff (or, equivalently, a similarity threshold) is applied uniformly to every query, regardless of whether the query is narrow and well-supported by a single passage or broad and supported by a constellation of weakly related cues. Third, multi-hop queries that require chaining evidence across passages are handled poorly by DPR: the most relevant single passage is rarely the one that bridges two facts, and iterative retrieval techniques add latency and complexity without restoring the structural information that DPR fundamentally discards.

### 2.2.3 Graph-Based Retrieval-Augmented Generation

Graph-based RAG methods address these limitations by introducing an explicit graph structure over the retrieval index. GraphRAG [11] prompts an LLM to extract entities and relations from each chunk of the corpus, aggregates the extracted triples into a global entity graph, and performs hierarchical community detection to summarize the graph at multiple resolutions. At inference time, GraphRAG retrieves a set of community summaries whose level of granularity is matched to the input query and feeds them to the generator. LightRAG [14] pursues a similar high-level architecture but emphasizes efficiency, replacing the heavy global community summarization step with a lighter local-then-global retrieval procedure that scales to large corpora.

These systems represent a substantial advance over flat dense retrieval. By exposing explicit entities and relations, they enable the language model to reason over a structured representation of the corpus, and by retrieving connected substructures rather than isolated passages, they support multi-hop queries more naturally. Empirical evaluations on knowledge-intensive QA benchmarks confirm that graph-based RAG outperforms strong DPR baselines, particularly on queries that require synthesizing information from many

documents.

However, current graph-based RAG systems suffer from three weaknesses that motivate the proposed work in this thesis. First, the graph construction step is dominated by LLM prompting, which is prone to hallucination: spurious entities and edges enter the graph and propagate downstream as silent errors. Second, the retrieval policies are largely heuristic—community summarization at a fixed resolution, top-k neighbor expansion, or similarity-based filtering—and lack a principled formulation of what evidence the language model actually needs. Third, the resulting retrieved context is often a disconnected set of nodes or a single shortest path, neither of which exposes the kind of coherent, multi-hop reasoning chain that downstream tasks require. The MCMI subgraph generation approach proposed in this thesis is designed precisely to fill this gap.

### 2.2.4 Comparison: Vector Retrieval versus Graph-Structured Retrieval

The contrast between vector retrieval and graph-based retrieval can be sharpened by considering three axes. First, on *representation*: dense retrieval represents each unit of evidence as an isolated vector, while graph-based retrieval represents evidence as nodes embedded in a graph with explicit relational structure. Second, on *retrieval primitive*: dense retrieval supports nearest-neighbor lookup, while graph-based retrieval supports nearest-neighbor lookup augmented by traversal, expansion, and subgraph extraction. Third, on *adaptivity*: dense retrieval is governed by a fixed top-k or threshold, while graph-based retrieval can in principle adapt the size and shape of the retrieved context to the query. In current practice, however, graph-based RAG systems often fail to exploit the third axis, applying fixed traversal depths or fixed community resolutions in a manner that mirrors the inflexibility of dense retrieval. Closing this gap—designing genuinely adaptive, structure-aware retrieval policies—is one of the central concerns of this thesis.

### 2.2.5 Current Limitations and Open Challenges

Beyond the specific weaknesses of current graph-based RAG systems, several broader challenges remain open. The construction of the underlying graph is rarely auditable: a downstream user of GraphRAG cannot easily inspect which spans of the corpus support a

given edge, nor can they correct hallucinated edges without re-running the entire pipeline. The matching of query subgraphs to the indexed graph is largely embedding-based, with little role for explicit path structure. And the systems-level cost of graph-based RAG is high: graph construction is expensive, and per-query traversal can rival or exceed the cost of generation itself, especially when the index is large. The proposed semantic subgraph matching with graph-aware caching directly targets the last of these challenges.

## 2.3 Few-Shot Text Classification

### 2.3.1 Traditional Text Classification Methods

Text classification has a long history that pre-dates deep learning, with classical methods relying on bag-of-words representations, TF-IDF features, and linear classifiers such as logistic regression and support vector machines. The advent of word embeddings and recurrent or convolutional neural networks pushed performance substantially higher on standard benchmarks, and the introduction of pre-trained language models such as BERT [10] and its successors set a new state of the art. Standard text classification, however, assumes a fixed label set with substantial labeled data per class, and the methods developed in this regime translate poorly to settings where labels evolve and supervision is scarce.

### 2.3.2 Few-Shot Learning Paradigms

Few-shot learning addresses the data-scarcity side of the problem. Prototypical networks [32] compute a prototype embedding for each class as the mean of its support examples and classify a query example by nearest-prototype assignment. Matching networks [37] generalize this idea by attending over the support set, while meta-learning approaches such as MAML [12] explicitly optimize for fast adaptation to new tasks. These methods perform well on few-shot benchmarks with stable label spaces but rest on assumptions that are violated in dynamic settings: the embedding space is trained on a fixed task distribution, and the support set is assumed to be known and complete at the time the prototypes or meta-parameters are estimated.

### 2.3.3 LLM-Based Few-Shot Classification

The emergence of large language models has reshaped few-shot text classification. In-context learning, popularized by GPT-3 [4], frames few-shot classification as a prompting problem: the model is given a small number of labeled exemplars in its context window and asked to predict the label of a query. This approach is impressively flexible—it requires no parameter updates and supports arbitrary new label sets—but it is sensitive to the choice of exemplars, the order in which they are presented, and the wording of the prompt. Subsequent work has explored automated demonstration selection, calibration of label biases, and structured prompting, but the underlying limitation persists: in-context classification provides no principled mechanism for retrieving the exemplars that are most informative for a particular query, and its accuracy degrades rapidly as the label set grows or evolves.

A complementary line of work fine-tunes LLMs on small per-class datasets using parameter-efficient methods such as LoRA. While effective for static label sets, these methods are ill-suited to settings in which new labels arrive continuously, because each new label triggers a fresh fine-tuning episode and complicates serving and versioning.

### 2.3.4 Dynamic Label Settings and Online Learning

A growing body of work explicitly targets the dynamic label setting, in which the set of classes evolves over time. Online and continual learning methods aim to incorporate new classes without catastrophic forgetting of previously seen classes. In the text classification context, this is particularly challenging because the cost of relabeling is high and the new classes are often defined only by a handful of seed examples. Few-shot continual learning combines the two challenges and exposes the limitations of both static few-shot methods and continual learning methods: prototypes drift, embeddings become stale, and the system has no principled mechanism to retrieve previously labeled exemplars that remain relevant under the new label structure.

Graph-based approaches are a natural fit for this setting because they can incorporate new labels and exemplars as new nodes without re-indexing the entire structure. The GORAG contribution of this thesis is built on this observation, but unlike prior graph-

based classification methods that rely on uniform indexing or non-adaptive retrieval, GORAG introduces information-theoretic edge weights and a minimum-cost spanning-tree retrieval procedure that adapts the retrieved evidence to the structure of each query.

### 2.3.5 Social Media Text Classification Applications

Social media classification provides a particularly stringent test bed for dynamic few-shot methods. Hate speech detection, misinformation identification, and abuse detection on platforms such as Twitter, Reddit, and Facebook face a perpetually evolving landscape of slang, evasion strategies, and platform-specific idiom. Public benchmarks for these tasks are quickly outdated, and the cost of producing high-quality labeled data at the pace of platform evolution is prohibitive. Surveys of the area document the heavy reliance of moderation pipelines on continual retraining, transfer from related tasks, and brittle keyword-based filters. Methods that can incorporate a handful of new exemplars and immediately retrieve them at inference time—without retraining—are especially valuable in this domain. GORAG is evaluated against this backdrop and demonstrates substantial gains over both static few-shot baselines and prior graph-based RAG approaches.

## 2.4 Graph Algorithms for Information Retrieval

### 2.4.1 Subgraph Matching

Subgraph matching is the problem of locating, within a target graph, all subgraphs isomorphic to a query graph. Exact matching algorithms such as VF2 [7] and its descendants traverse the candidate space using carefully designed feasibility checks that prune unpromising branches early. Modern variants such as ISMAGS [16] exploit symmetries to accelerate matching on large graphs, while approximate matching methods relax the strict isomorphism requirement to allow noisy or partial matches.

In the context of information retrieval and RAG, exact subgraph matching is rarely the right primitive. Queries and indexed graphs are described in different vocabularies, edges may be paraphrased rather than identical, and the goal is typically to find semantically equivalent substructures rather than literal copies. This motivates path-aware se-

mantic subgraph matching, which generalizes the matching primitive to allow path-level semantic equivalence between query and target while retaining the algorithmic backbone of classical methods. The proposed semantic subgraph matching contribution of this thesis builds on this generalization.

### 2.4.2 Personalized PageRank and Influence Propagation

PageRank and its personalized variant assign a relevance score to each node in a graph, with personalized PageRank biasing the random-walk distribution toward a query-specified seed set. Efficient algorithms for personalized PageRank, including bidirectional estimators [25] and Monte Carlo methods [3], have made personalized PageRank computationally tractable on graphs with hundreds of millions of nodes. These algorithms underpin a wide range of applications, from recommendation to community detection to graph-based learning to rank.

Personalized PageRank is also a natural retrieval primitive in graph-based RAG: given a query mapped to a set of seed nodes, the personalized PageRank vector identifies the most influential nodes in the indexed graph relative to those seeds. The MCMI subgraph generation problem proposed in this thesis can be viewed as a structured generalization of personalized PageRank, in which influence is jointly maximized with structural connectivity under a cost budget, producing a coherent subgraph rather than a flat ranking.

### 2.4.3 Minimum Spanning Tree and Its Variants

Minimum spanning tree (MST) algorithms compute, for a connected weighted graph, a spanning subgraph of minimum total edge weight. Classical algorithms by Kruskal and Prim run in near-linear time, and Steiner-tree variants extend the formulation to require connectivity only over a designated subset of terminals, at the cost of NP-hardness and the introduction of approximation algorithms. MST and Steiner-tree formulations are widely used in network design, clustering, and visualization.

In information retrieval, MST formulations are valuable because they expose a notion of *adaptive* evidence selection: rather than retrieving a fixed top-k neighborhood, an MST-style retriever returns whatever subgraph is necessary to connect the query to the most

informative anchors, with the tree size determined by the structure of the underlying graph. GORAG leverages this property, retrieving a query-specific tree from a heterogeneous label-keyword-exemplar graph with edge weights that reflect the discriminative power of each connection. The proposed advanced graph-based RAG framework generalizes the MST idea further, replacing pure cost minimization with a joint MCMI criterion that balances cost and influence under a budget constraint.

### 2.4.4 Graph-Based Reasoning and Inference

Beyond retrieval, graph algorithms also underlie a range of approaches to reasoning. Graph neural networks aggregate information from node neighborhoods to support node and edge prediction; logical reasoning over KGs has been formalized through path-ranking, embedding-based reasoning, and rule mining; and explicit symbolic reasoners traverse multi-hop paths in KGs to answer compositional queries. The interface between these graph-based reasoners and modern language models is an active research frontier, with recent surveys [27] cataloguing both the opportunities and the challenges. The proposed work in this thesis sits at this interface: rather than replacing language model reasoning with graph-based inference or vice versa, it equips language models with explicit graph-derived evidence and explicit reasoning paths, allowing each component to play to its strengths.

## 2.5 Summary and Research Gaps

### 2.5.1 Comparative Summary of Existing Approaches

The four bodies of literature surveyed in this chapter approach knowledge-grounded language understanding from different angles: KG-based methods bring explicit structure but limited flexibility, deep-learning-based methods bring flexibility but limited grounding, RAG methods bring external evidence but often discard relational structure, and graph algorithms bring principled retrieval primitives but have not yet been systematically applied to RAG with the right combination of adaptivity and efficiency. Table 2.1 summarizes the principal axes along which these approaches differ.

Table 2.1: Comparison of existing approaches and the four contributions of this thesis along five capability axes: external knowledge integration, adaptive retrieval, explicit reasoning chains, dynamic label support, and efficient subgraph matching. ✓indicates that the capability is supported, ~ indicates partial support, and × indicates no support.

| Approach | External KG Integration | Adaptive Retrieval | Reasoning Chains | Dynamic Labels | Efficient Matching |
|---|---|---|---|---|---|
| KG-based CTA [2] | ✓ | × | ~ | × | × |
| DL-based CTA [10] | × | × | × | × | × |
| DPR [22] | ~ | × | × | × | N/A |
| GraphRAG [11] | ✓ | ~ | ~ | × | × |
| LightRAG [14] | ✓ | ~ | ~ | × | ~ |
| Prototypical Net [32] | × | × | × | ~ | N/A |
| LLM In-context [4] | ~ | × | × | ~ | N/A |
| KGLink (Contrib. 1) [40] | ✓ | ~ | ~ | × | N/A |
| GORAG (Contrib. 2) [39] | ~ | ✓ | ✓ | ✓ | ~ |
| Adv. Graph-RAG (Contrib. 3) | ✓ | ✓ | ✓ | ~ | ~ |
| Sem. Subgraph Match (Contrib. 4) | ✓ | ✓ | ✓ | ~ | ✓ |

The table makes explicit that no existing approach jointly supports the five capabilities that structural augmented reasoning requires. KG-based methods integrate external knowledge but lack adaptive retrieval and dynamic-label support. Deep-learning-based methods offer neither external knowledge nor adaptive retrieval. DPR provides only a coarse approximation of external knowledge through retrieved passages and applies a uniform retrieval policy. Graph-based RAG systems such as GraphRAG and LightRAG move closer to the ideal but fall short on adaptivity, on the explicitness of reasoning chains, and on dynamic-label support. Few-shot methods such as prototypical networks and LLM in-context learning support dynamic labels only weakly and ignore external knowledge structure entirely.

### 2.5.2 Specific Research Gaps Addressed by This Thesis

The comparison in Table 2.1 highlights four specific research gaps that this thesis fills.

**Gap 1: No hybrid KG and PLM method jointly addresses granularity and missing context for column type annotation.** As discussed in Section 2.1, KG-based CTA suffers from granularity mismatch and brittleness on out-of-KG entities, while DL-based CTA

suffers from missing-context failures on tables with sparse explicit signal. No prior work combines the two evidence sources through a granularity-aware reconciliation mechanism that adapts to per-column reliability. KGLink (Contribution 1) closes this gap.

**Gap 2: Existing graph-based RAG lacks adaptive, input-specific retrieval for dynamic few-shot classification.** As discussed in Sections 2.2 and 2.3, current graph-based RAG systems apply fixed retrieval policies and assume static label sets, while existing few-shot methods ignore the relational structure of labeled exemplars. No prior work organizes labels, keywords, and exemplars into a heterogeneous weighted graph with information-theoretic edges and retrieves a query-specific subgraph through a principled MST formulation. GORAG (Contribution 2) closes this gap.

**Gap 3: No principled graph algorithm produces explicit reasoning chains for general-purpose graph-based RAG.** As discussed in Sections 2.2 and 2.4, existing graph-based RAG systems rely on LLM-driven graph construction prone to hallucination and on heuristic retrieval policies that return disconnected node sets rather than coherent reasoning chains. No prior work formalizes evidence selection as a minimum-cost maximum-influence subgraph generation problem with bounded approximation guarantees. The proposed advanced graph-based RAG framework (Contribution 3) closes this gap.

**Gap 4: Semantic subgraph matching has not been applied to RAG retrieval with graph-aware caching.** As discussed in Section 2.4, classical subgraph matching algorithms target exact structural isomorphism and are too rigid for the semantic mismatches typical between user queries and indexed graphs. No prior work generalizes subgraph matching to path-level semantic equivalence and pairs it with a graph-aware key–value cache that exploits the structural overlap between successive queries to reuse retrieval and inference computations. The proposed semantic subgraph matching framework (Contribution 4) closes this gap.

These four gaps motivate the four contributions of this thesis. Subsequent chapters develop each contribution in detail and report—or, in the case of the proposed contributions, plan—the empirical evaluations that validate the corresponding designs.

# CHAPTER 3

# KGLINK: KNOWLEDGE GRAPH ENHANCED COLUMN TYPE ANNOTATION

## 3.1 Introduction

Tabular data on the Web and in enterprise data lakes constitute one of the most ubiquitous formats for structured information. A central prerequisite for downstream tasks such as table search, table augmentation, data integration, and question answering is to understand the *semantics* of each column, i.e., to identify what real-world concept the column denotes. This task is commonly referred to as *Column Type Annotation* (CTA). For instance, given a column whose cells are `{"LeBron James", "Stephen Curry", "Kevin Durant"}`, a CTA system should assign a semantic label such as Basketball_Player drawn from a target type vocabulary.

Two complementary lines of research address CTA. The first line exploits large-scale knowledge graphs (KGs) such as Wikidata [38] and DBpedia [2], by linking cell values to KG entities and aggregating the types associated with those entities [15]. The second line, driven by recent advances in pre-trained language models (PLMs) [10], treats CTA as a multi-class classification problem on top of contextual representations of the table [17, 9, 33]. Although both directions have produced strong results on standard benchmarks, neither is fully satisfactory in practice: KG-based methods struggle to align the granularity of KG types with that of the dataset's label set, while PLM-based methods are limited by the information that is locally observable inside a single table.

In this chapter, we present **KGLink** [40], a hybrid CTA framework that systematically combines knowledge graphs with pre-trained language models in order to overcome the limitations of either family in isolation. KGLink addresses two long-standing challenges that we explicitly identify and analyze: the *type granularity problem* and the *missing valuable context problem*. The remainder of this chapter formalizes the problem (Sec. 3.2), presents the framework (Sec. 3.3), discusses our solutions to the two challenges

(Sec. 3.4 and Sec. 3.5), reports an extensive empirical evaluation (Sec. 3.6), and concludes with a short summary (Sec. 3.7). KGLink constitutes the first work of this thesis and connects naturally to the subsequent chapters that further investigate trustworthy graph-based learning.

## 3.2 Problem Formulation

We begin by introducing the notation that will be used throughout this chapter and giving a formal definition of the column type annotation task. We then illustrate, through a concrete example, the two challenges that motivate the design of KGLink.

### 3.2.1 Notation and Task Definition

A relational table is denoted by $\mathsf{T} = (\mathbf{h}, \mathbf{C})$, where $\mathbf{h} = \langle h_1, h_2, \ldots, h_n \rangle$ is the (possibly empty) header row and $\mathbf{C} = \langle c_1, c_2, \ldots, c_n \rangle$ is the list of $n$ columns. Each column $c_j$ contains a sequence of cell values $c_j = \langle v_{1,j}, v_{2,j}, \ldots, v_{m,j} \rangle$ drawn from a (possibly heterogeneous) value space. The header $h_j$ associated with $c_j$ is treated as additional contextual evidence rather than as a ground-truth label, since headers are frequently missing or noisy in real tables.

A type vocabulary is denoted by $\mathcal{Y}$ and consists of a finite set of semantic types, e.g., Person, Athlete, Basketball_Player, Country. The type vocabulary is typically derived from the type hierarchy of a KG $\mathcal{G} = (\mathcal{E}, \mathcal{T}, \mathcal{R})$, where $\mathcal{E}$ is the entity set, $\mathcal{T}$ is the type set (a directed acyclic graph induced by the `subClassOf` relation), and $\mathcal{R}$ encodes the relationships and the entity–type assignment $\tau : \mathcal{E} \to 2^{\mathcal{T}}$. For each entity $e \in \mathcal{E}$, $\tau(e)$ returns the set of types declared in $\mathcal{G}$.

**Definition 3.2.1 (Column Type Annotation)** *Given a table $\mathsf{T} = (\mathbf{h}, \mathbf{C})$, a target type vocabulary $\mathcal{Y} \subseteq \mathcal{T}$, and a knowledge graph $\mathcal{G}$, the goal of column type annotation is to learn a function $f_\theta : c_j \mapsto y_j \in \mathcal{Y}$ that, for each column $c_j$, assigns a single semantic label $y_j$ that best describes the values in $c_j$ within the context provided by the remaining columns and the header.*

In Definition 3.2.1, we assume that $\mathcal{Y}$ is a subset of the KG type set rather than a flat




Figure 3.1: An illustrative example of column type annotation. Figure (a) has the knowledge missing issue. Column one has the ground truth column type as *Year*. However, it is hard for pure knowledge graph based methods to link numeric columns to the knowledge graph. Figure (b) has valuable context missing issues, and it is hard for the deep learning based model to carry out column annotation tasks. The target column's context information, the column two and three, are not helpful for the column type annotation task on column one since they could not provide any information close to the ground truth label: *Cricketer*.

vocabulary; this is consistent with widely used CTA benchmarks such as the SemTab challenge datasets [21]. We focus on the single-label setting in this chapter, although KGLink can be straightforwardly extended to the multi-label setting by replacing the final softmax with a sigmoid layer.

### 3.2.2 An Illustrative Example

Consider the simple table shown in Fig. 3.2.2, with two columns: $c_1$ contains a list of person names (`LeBron James`, `Stephen Curry`, `Kevin Durant`) and $c_2$ contains numeric values (`2.06`, `1.88`, `2.11`). We highlight two challenges that any practical CTA system must confront on tables of this kind.

**Challenge 1: Type Granularity.** Linking the cells of $c_1$ to a KG yields entities all of which share multiple valid types, including Person, Athlete, and Basketball_Player. A KG-only method that simply takes the most frequent or most specific type may not match the granularity expected by the downstream application: when the target vocabulary $\mathcal{Y}$ contains both Athlete and Basketball_Player, only the latter is correct; when $\mathcal{Y}$ contains only Athlete, the former should be returned. The granularity that is appropriate for a given dataset cannot be inferred from the KG alone.

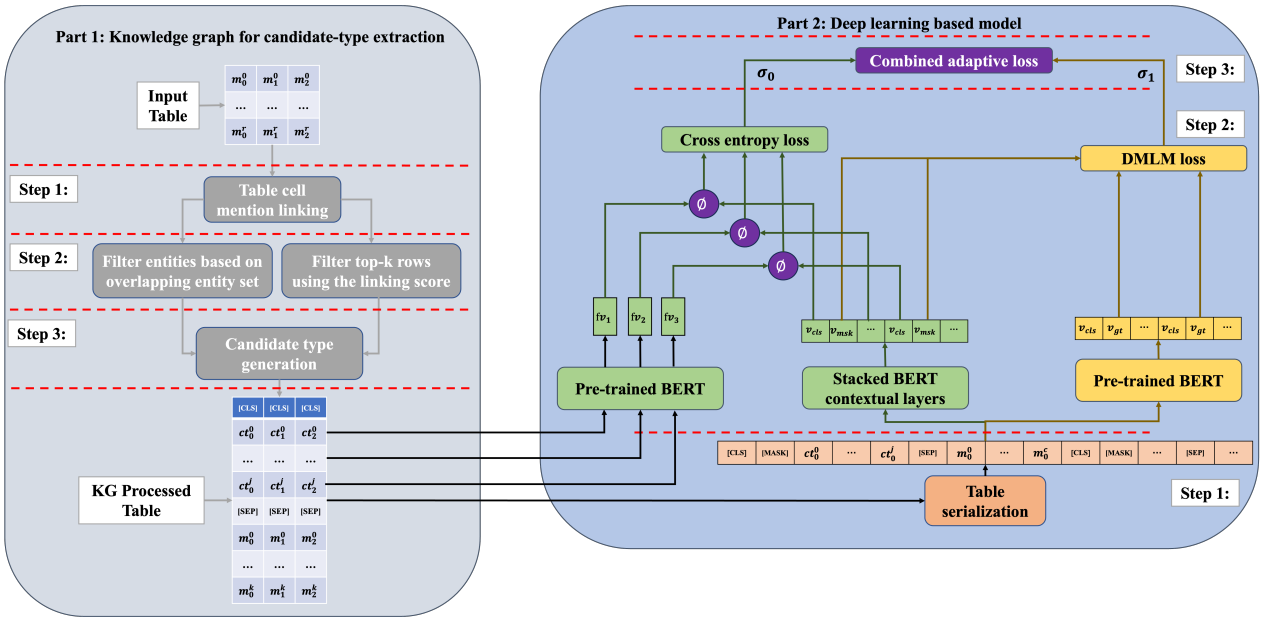



Figure 3.2: High-level architecture of KGLink. The KG linking module produces a candidate type distribution from external knowledge, while the PLM encoding module captures contextual signals from the table itself; a hybrid reasoning module fuses the two sources to produce the final prediction.

**Challenge 2: Missing Valuable Context.** The numeric column $c_2$ contains values that, considered in isolation, are compatible with many types: Height, Distance (m), Latitude, etc. The cells in $c_2$ themselves do not provide enough textual signal for a PLM to disambiguate between these candidates. The valuable context that would resolve the ambiguity, namely that the column appears next to a column of basketball players, is implicit and may not be expressible through standard table serialization techniques. A pure PLM-based classifier, lacking access to external world knowledge, often errs on exactly these columns.

These two challenges, which we have observed empirically on standard CTA benchmarks, motivate the hybrid design of KGLink described in the next section.

## 3.3 The KGLink Framework

### 3.3.1 Overview

KGLink is composed of three tightly coupled modules, illustrated in Fig. 3.2. Given an input column $c_j$ together with its surrounding table T, the framework proceeds in three

steps. First, the *Knowledge Graph Linking Module* (Sec. 3.3.2) maps cell values to KG entities, collects the types of the linked entities, and aggregates them into a column-level candidate type distribution $p_{KG}(y \mid c_j)$. This distribution carries explicit external knowledge about what the column *can* be. Second, the *Pre-trained Language Model Encoding Module* (Sec. 3.3.3) encodes the column together with its header and neighboring columns through a serialized table representation, producing a contextual distribution $p_{PLM}(y \mid c_j, T)$. This distribution captures the discriminative signal that arises from the table's local context. Third, the *Hybrid Reasoning Module* (Sec. 3.3.4) combines the two distributions by attending over the KG type hierarchy, resolving granularity conflicts and supplying external context to the PLM where the table alone is insufficient. The final prediction is given by

$$\hat{y}_j = \arg\max_{y \in \mathcal{Y}} \; \Phi\big(p_{KG}(y \mid c_j),\; p_{PLM}(y \mid c_j, T),\; \mathcal{H}_{KG}\big), \tag{3.1}$$

where $\mathcal{H}_{KG}$ denotes the type hierarchy of the KG and $\Phi(\cdot)$ is the hybrid reasoning function described in Sec. 3.3.4.

### 3.3.2 Knowledge Graph Linking Module

The KG linking module is responsible for converting the cells of $c_j$ into a set of typed entities and aggregating the resulting types into a column-level candidate distribution. Two technical challenges arise: (i) cells are noisy strings (e.g., `LeBron James Jr.`) or numeric values, and may not literally match any KG label; and (ii) entities have multiple types at different granularity levels, so we need a principled way to combine them.

**Entity Linking for String Columns.** For a string-valued cell $v_{i,j}$, we obtain a candidate set of linked entities by

$$\mathcal{E}_{i,j} \;=\; \mathrm{TopK}\big(\{e \in \mathcal{E} \mid \mathrm{score}_{link}(v_{i,j}, e) \geqslant \tau_{link}\}\big), \tag{3.2}$$

where $\mathrm{score}_{link}$ is a string-matching score that combines lexical similarity (e.g., the Jaccard similarity on character $n$-grams), entity popularity, and contextual disambiguation features computed using the surrounding cells in the same row. The threshold $\tau_{link}$ filters out unreliable links, and the top-$K$ truncation keeps only the most plausible candidates per cell. Each linked entity $e$ contributes its declared type set $\tau(e) \subseteq \mathcal{T}$, possibly weighted by $\mathrm{score}_{link}(v_{i,j}, e)$.

**Linking for Numeric Columns.** Numeric cells require a different treatment because they cannot be matched against entity surface forms directly. KGLink instead exploits the value distribution of numeric attributes already known in the KG. For each numeric type $y \in \mathcal{Y}$, the KG provides a set of values $\mathcal{V}_y$ extracted from the corresponding attribute (e.g., height for Person). The KG-derived score of a numeric column with respect to type $y$ is computed by comparing the empirical distribution of cells in $c_j$ with $\mathcal{V}_y$ via a divergence measure $D(\cdot, \cdot)$:

$$\text{score}_{\text{num}}(c_j, y) \;=\; \exp\big(-D(\hat{P}_{c_j},\, \hat{P}_{\mathcal{V}_y})\big), \tag{3.3}$$

where $\hat{P}_{c_j}$ and $\hat{P}_{\mathcal{V}_y}$ denote the empirical distributions over the column values and the KG-attribute values, respectively. Concretely, we adopt the Jensen–Shannon divergence on histograms with shared bins, which we found to be more robust to outliers than the Kolmogorov–Smirnov statistic.

**Type Aggregation.** Given the per-cell evidence above, the KG linking module aggregates types across all cells of column $c_j$ to produce $p_{KG}(y \mid c_j)$. Let $w_{i,j,e}$ denote the weight of entity $e$ for cell $v_{i,j}$ (zero for unlinked entities). The column-level evidence for a type $y \in \mathcal{Y}$ is

$$s_{KG}(y \mid c_j) \;=\; \frac{1}{m} \sum_{i=1}^{m} \sum_{e \in \mathcal{E}_{i,j}} w_{i,j,e} \cdot \mathbb{1}\big[y \in \bar{\tau}(e)\big], \tag{3.4}$$

where $\bar{\tau}(e)$ denotes the set of types of $e$ together with all of their ancestors in the type hierarchy. The use of $\bar{\tau}$ rather than $\tau$ is essential: it allows a more specific entity type, e.g., Basketball_Player, to also contribute evidence for ancestor types such as Athlete. The final distribution $p_{KG}(y \mid c_j) \propto s_{KG}(y \mid c_j)$ is then obtained by normalizing $s_{KG}$ over $\mathcal{Y}$.

**Granularity-Aware Type Selection.** Equation (3.4) alone does not solve the granularity problem: aggregating over ancestors gives many types non-zero support, and choosing the most specific type does not always match the dataset granularity. KGLink therefore defers granularity selection to the hybrid reasoning module, where evidence from the table context narrows down the appropriate level of the hierarchy. This decoupling is one of the key design choices of KGLink, and we discuss it in detail in Sec. 3.4.

### 3.3.3 Pre-trained Language Model Encoding Module

The PLM encoding module captures contextual signals that the KG cannot easily provide, such as the joint co-occurrence of column values, header tokens, and neighboring columns. We build on the encoder-only architecture of BERT [10] and follow a serialization scheme similar in spirit to recent table-aware PLMs [9, 18, 33].

**Table Serialization.** Given a table $T$ and a target column $c_j$, the input string fed to the PLM is

$$\begin{aligned} \mathcal{S}(T, c_j) \;=\; & [\textsc{cls}]\,\mathrm{tok}(h_j)\,[\textsc{tgt}]\,\mathrm{tok}(v_{1,j})\,[\textsc{sep}]\;\ldots\,[\textsc{sep}]\,\mathrm{tok}(v_{m',j}) \\ & [\textsc{ctx}]\,\bigoplus_{k \neq j} \big(\mathrm{tok}(h_k)\,[\textsc{val}]\,\mathrm{tok}(v_{1,k})\;\ldots\big), \end{aligned} \tag{3.5}$$

where $\mathrm{tok}(\cdot)$ denotes the WordPiece tokenizer of the PLM, $m'$ is the number of cell values sampled from $c_j$ when the column exceeds the maximum sequence length, and [TGT], [CTX], [VAL] are special marker tokens that distinguish target column tokens, contextual column tokens, and individual cell boundaries. The [CLS] representation produced by the encoder is taken as the column-level embedding $\mathbf{z}_j \in \mathbb{R}^d$.

**Classification Head and Fine-Tuning.** The PLM-based score for a type $y \in \mathcal{Y}$ is given by a linear projection followed by softmax:

$$p_{\mathrm{PLM}}(y \mid c_j, T) \;=\; \mathrm{softmax}\big(\mathbf{W}_{\mathrm{cls}}\,\mathbf{z}_j + \mathbf{b}_{\mathrm{cls}}\big)_y, \tag{3.6}$$

where $\mathbf{W}_{\mathrm{cls}} \in \mathbb{R}^{|\mathcal{Y}| \times d}$ and $\mathbf{b}_{\mathrm{cls}} \in \mathbb{R}^{|\mathcal{Y}|}$ are learnable parameters. We fine-tune the entire encoder together with the classification head using a standard cross-entropy loss

$$\mathcal{L}_{\mathrm{PLM}} \;=\; -\frac{1}{|\mathcal{D}|} \sum_{(c_j, y_j^\star) \in \mathcal{D}} \log p_{\mathrm{PLM}}(y_j^\star \mid c_j, T), \tag{3.7}$$

where $\mathcal{D}$ is the training set of labeled columns and $y_j^\star \in \mathcal{Y}$ is the ground-truth type of column $c_j$. To improve robustness, we apply two regularization techniques during fine-tuning: (i) cell sampling, where $m' \leqslant m$ values are randomly sampled at each training step; and (ii) header dropout, where $h_j$ is replaced by an empty token with probability $\rho_h$, forcing the model to rely on the cells when the header is missing.

### 3.3.4 Hybrid Reasoning and Type Prediction

The two modules described above produce two complementary distributions over $\mathcal{Y}$: $p_{KG}(\cdot \mid c_j)$ encodes external world knowledge, while $p_{PLM}(\cdot \mid c_j, T)$ encodes table-internal context. The hybrid reasoning module of KGLink combines them while explicitly respecting the structure of the type hierarchy.

**Hierarchy-Aware Fusion.** Let $\mathcal{H}_{KG} = (\mathcal{Y}, E_h)$ denote the hierarchy restricted to the target vocabulary, where each edge in $E_h$ points from a child type to its parent. We propagate PLM-derived evidence through the hierarchy by computing

$$\tilde{p}_{PLM}(y \mid c_j, T) \;=\; p_{PLM}(y \mid c_j, T) + \beta \sum_{y' \in \mathrm{Anc}(y)} p_{PLM}(y' \mid c_j, T), \tag{3.8}$$

where $\mathrm{Anc}(y)$ denotes the ancestors of $y$ in $\mathcal{H}_{KG}$ and $\beta \in (0, 1)$ is a damping factor. This step lets the PLM's evidence at coarse granularity (e.g., Person) sharpen the support for compatible fine-grained types (e.g., Basketball_Player) and vice versa.

The final hybrid score is then a convex combination of the two sources, modulated by a gating term that depends on the confidence of each module:

$$\Phi(y) \;=\; \alpha(c_j, T) \cdot \tilde{p}_{PLM}(y \mid c_j, T) + \big(1 - \alpha(c_j, T)\big) \cdot p_{KG}(y \mid c_j), \tag{3.9}$$

where the gate $\alpha(c_j, T) \in [0, 1]$ is produced by a small feed-forward network from features such as the entropy of $p_{PLM}$, the link coverage of the column (i.e., the fraction of cells with at least one linked entity), and the depth of the most specific KG type with non-trivial support. When PLM confidence is high and KG coverage is low, $\alpha$ is close to one, and the model behaves as a PLM-only classifier; when KG evidence is strong and PLM confidence is low, $\alpha$ is close to zero, and the model leans toward the KG. The classifier in Eq. (3.1) reduces to selecting the type with the highest fused score $\Phi(y)$ subject to the granularity constraints discussed next.

## 3.4 Addressing Type Granularity

We now revisit the type granularity challenge introduced in Sec. 3.2.2 and explain how KGLink resolves it through the joint use of the KG hierarchy and PLM evidence.

### 3.4.1 Why Granularity Is Hard for KG-only Methods

A purely KG-based annotator that selects the most popular or the most specific type per column tends to be miscalibrated with respect to the dataset's label set $\mathcal{Y}$. Consider again the column of basketball players in Fig. 3.2.2. Linking these cells to Wikidata yields entities each of which is annotated with many types, including Q5 (Person), Q937857 (Athlete), Q3665646 (Basketball_Player), and so on. If $\mathcal{Y}$ contains both Athlete and Basketball_Player, the most specific KG type is the right answer. However, if $\mathcal{Y}$ contains only Athlete but not Basketball_Player, then the most specific type is out-of-vocabulary and the KG-only method must back off to an ancestor whose level depends on the dataset. The choice of this level is dataset-dependent and not encoded in the KG itself.

### 3.4.2 KGLink's Solution: Hierarchy-Constrained Selection

KGLink resolves the granularity issue by combining three sources of information when selecting the predicted type for column $c_j$. First, the candidate set is restricted to types that are present in the target vocabulary $\mathcal{Y}$. Second, among these candidates, the KG aggregation in Eq. (3.4) provides the cumulative evidence based on entity types and their ancestors. Third, the hierarchy-aware PLM evidence $\tilde{p}_{\mathrm{PLM}}(y \mid c_j, T)$ provides a soft preference that depends on the actual labels seen during training.

We formalize the granularity-constrained selection as follows. Let $\mathcal{Y}^{\star}_{c_j} = \{y \in \mathcal{Y} \mid s_{\mathrm{KG}}(y \mid c_j) \geqslant \tau_y\}$ be the set of types that have non-trivial KG support, where $\tau_y$ is a small threshold. KGLink solves

$$\hat{y}_j \;=\; \arg\max_{y \in \mathcal{Y}^{\star}_{c_j}} \; \Phi(y) \quad \text{s.t.} \quad \nexists\, y' \in \mathcal{Y}^{\star}_{c_j} : y' \sqsubset y \;\wedge\; \Phi(y') \geqslant \gamma\, \Phi(y), \tag{3.10}$$

where $y' \sqsubset y$ means that $y'$ is a strict descendant of $y$ in $\mathcal{H}_{\mathrm{KG}}$, and $\gamma \in (0, 1]$ is a granularity preference factor. Intuitively, KGLink prefers a more specific type whenever the descendant's fused score is comparable to that of an ancestor; otherwise it falls back to the ancestor. This criterion adapts to the dataset granularity because the PLM has been trained on the dataset's label distribution and tends to assign higher probability mass at the level the dataset prefers.

### 3.4.3 Theoretical Justification

The behavior of Eq. (3.10) can be motivated by the following observation.

**Proposition 1 (Granularity Consistency)** *Assume that the PLM is well-calibrated on the training distribution of $\mathcal{Y}$, and that the KG aggregation $s_{KG}$ is monotone with respect to the hierarchy, i.e., $s_{KG}(y \mid c_j) \leqslant s_{KG}(y' \mid c_j)$ whenever $y' \in \mathrm{Anc}(y)$. Then the prediction $\hat{y}_j$ defined in Eq. (3.10) converges, as the size of the training set grows, to the most specific type in $\mathcal{Y}$ that is consistent with the empirical distribution of column $c_j$.*

A sketch of the argument is as follows. The monotonicity of $s_{KG}$ ensures that ancestors are never weaker than their descendants under the KG. Calibration of the PLM ensures that $\tilde{p}_{PLM}$ concentrates around the granularity level that dominates the training set. The constraint in Eq. (3.10) then prefers the deepest type whose fused score remains within a factor of $\gamma$ of the best ancestor, recovering the dataset-preferred granularity in the limit. A more rigorous statement and proof are deferred to future work.

## 3.5 Addressing Missing Context

The second challenge identified in Sec. 3.2.2 is that of *missing valuable context*: situations in which the table-internal information is insufficient to determine the column type and external knowledge is required.

### 3.5.1 When Does Context Go Missing?

Through preliminary studies on standard CTA benchmarks, we identify three recurring patterns of missing context that hurt the performance of pure DL-based annotators:

- **Headerless or noisy headers.** Many real tables either omit headers or use abbreviations and codes (e.g., `Ht`, `NLOC`) that the PLM has rarely seen during pre-training. Without the header, ambiguous string columns become much harder to disambiguate.

- **Numeric columns.** As discussed earlier, numeric cells alone do not provide enough textual signal for a PLM. The decisive evidence is statistical (the value distribution) rather than lexical, and it lies outside the table.
- **Short tables.** For tables with very few rows ($m \leqslant 3$), the PLM observes too few cells to estimate a column's semantic type reliably. Each cell becomes disproportionately influential, amplifying lexical noise.

### 3.5.2 Knowledge Graphs as External Context

KGLink supplies the missing context through the KG. For string columns, even a single cell with a high-confidence link injects an entire entity profile into the candidate distribution $p_{KG}$, including types and attribute values that indirectly characterize the column. For numeric columns, the distributional matching against KG attribute values offers a strong signal that is independent of any cell text. The hybrid reasoning module then leans on the KG via the gate $\alpha(c_j, T)$ in Eq. (3.9): when PLM entropy is high or the header is missing, $\alpha$ shrinks toward zero, transferring the prediction responsibility to the KG.

### 3.5.3 Integration Mechanism

The integration mechanism that enables this transfer can be summarized as follows. We first compute three quantities that characterize the available context: the PLM entropy $H(p_{PLM}) = -\sum_y p_{PLM}(y) \log p_{PLM}(y)$, the KG link coverage $\mathrm{cov}(c_j) = \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}\big[\mathcal{E}_{i,j} \neq \emptyset\big]$, and the hierarchy depth $d^\star(c_j)$ of the most specific KG-supported type. The gate is then parameterized as

$$\alpha(c_j, T) \;=\; \sigma\big(\mathbf{w}^\top [\, H(p_{PLM}),\, \mathrm{cov}(c_j),\, d^\star(c_j)\,] + b\big), \tag{3.11}$$

where $\sigma(\cdot)$ is the sigmoid function and $(\mathbf{w}, b)$ are jointly trained with the PLM head. This makes the gate *adaptive*: it learns from data when to trust the PLM and when to trust the KG, rather than relying on a fixed rule. Empirically, we observe that $\alpha$ becomes small for numeric columns and short tables, confirming that the gate captures the intended notion of missing context.

Table 3.1: Statistics of the two CTA benchmarks used in our experiments.

| | **SemTab** | **VizNet** |
|---|---|---|
| **#Tables** | 3,048 | 32,265 |
| **#Columns** | 7,587 | 73,034 |
| **#Types** | 275 | 77 |
| **Avg. #Rows** | 69 | 20 |
| **Avg. #Columns** | 4.5 | 2.3 |
| **KG** | DBpedia | – |

## 3.6 Experiments

We empirically evaluate KGLink to answer the following questions. **(Q1)** Does KGLink outperform existing KG-only and DL-only methods on standard CTA benchmarks? **(Q2)** Does KGLink effectively address the type granularity problem? **(Q3)** Does KGLink benefit from external KG context when the table itself is uninformative? **(Q4)** What is the contribution of each module of KGLink?

### 3.6.1 Experimental Setup

**Datasets.** We follow the standard evaluation protocol of CTA and conduct experiments on two widely used benchmarks. **SemTab** [21] is the dataset released by the SemTab challenge series, containing thousands of tables annotated against the DBpedia type hierarchy at fine granularity. **VizNet** is a large-scale corpus of Web tables harvested from open data portals and visualization galleries, providing a complementary evaluation setting that is dominated by short tables without an explicit KG type hierarchy. The statistics of the two datasets are summarized in Tab. 3.1. The type vocabulary of each dataset is constructed from the labels that accompany the benchmark, and, for SemTab, intersected with the DBpedia hierarchy.

**Evaluation Metrics.** Following prior work, we report *Accuracy* and *Weighted-F1* over all annotated columns, which jointly capture overall correctness and class-imbalance-aware performance. For SemTab, we additionally disaggregate accuracy on numeric and non-numeric columns to assess performance on heterogeneous column types.

**Baselines.** We compare KGLink against three categories of baselines that cover the most representative CTA approaches in the literature. **KG-only and shallow methods. T2K Match** [29] links table values to DBpedia and aggregates types via weighted voting. **HNN** is a hybrid neural network classifier that combines character- and word-level features over column values. **Sherlock** [17] is a deep classifier that uses around 1,500 hand-crafted column statistics. **PLM-based methods. TURL** [9] is a transformer-based table encoder pre-trained on a large Web table corpus. **Doduo** [33] fine-tunes BERT on serialized tables for column type and column relation annotation. **TaBERT** jointly encodes natural-language utterances and tables for table understanding. **KG-augmented and contrastive methods. ColNet** [6] combines a CNN-based encoder with KG features. **RECA** aligns column representations with related-column context for type annotation. **Sudowoodo** adopts contrastive learning over column representations. We further include three ablated variants of KGLink itself, **KGLink w/o msk**, **KGLink w/o ct**, and **KGLink w/o fv**, which respectively disable the column masking objective, the contextual table serialization, and the KG-derived feature vectors; these variants serve as fine-grained ablations of the full model.

**Implementation Details.** We use the BERT-base-uncased model [10] and the larger DeBERTa as PLM backbones and fine-tune each for 5 epochs with the AdamW optimizer at a learning rate of $2 \times 10^{-5}$ and a batch size of 32. For entity linking, we use the public Wikidata and DBpedia dumps and a sparse retrieval index with character $n$-gram features ($n \in \{2, 3\}$). The hyperparameters $\beta$, $\gamma$, $\rho_h$, $K$, $\tau_{\text{link}}$, and $\tau_y$ are selected on the validation split via grid search. All experiments are run on a machine equipped with two NVIDIA A100 GPUs (80 GB), and we report results averaged over 3 random seeds.

### 3.6.2 Main Results

The main results are reported in Tab. 3.2. We make several observations.

First, the strongest configuration KGLink (DeBERTa) achieves the best Accuracy and Weighted-F1 on both benchmarks (87.24/85.81 on SemTab and 96.98/96.37 on VizNet), improving the best non-KGLink baseline (Doduo) by 2.57 Accuracy points on SemTab and by 1.53 Accuracy points on VizNet. Replacing the DeBERTa backbone with BERT-base still

Table 3.2: Column type annotation results on SemTab and VizNet, reporting Accuracy and Weighted-F1 (%). The best result in each column is in **bold**; the second-best is underlined. "–" indicates that the corresponding method is not applicable or not evaluated on that benchmark.

| | **SemTab** | | **VizNet** | |
|---|---|---|---|---|
| | Accuracy | Weighted-F1 | Accuracy | Weighted-F1 |
| **Sherlock** [17] | – | – | 94.21 | 93.85 |
| **T2K Match** [29] | 76.34 | 74.89 | – | – |
| **HNN** | 79.56 | 77.82 | 93.45 | 93.10 |
| **ColNet** [6] | 80.12 | 78.45 | – | – |
| **TaBERT** | 82.89 | 81.23 | 94.87 | 94.52 |
| **TURL** [9] | 83.45 | 81.92 | 95.12 | 94.78 |
| **Sudowoodo** | 83.78 | 82.15 | 95.21 | 94.86 |
| **RECA** | 84.23 | 82.67 | 95.33 | 94.98 |
| **Doduo** [33] | 84.67 | 83.10 | 95.45 | 95.11 |
| **KGLink w/o msk** | 86.14 | 84.54 | 95.95 | 95.67 |
| **KGLink w/o ct** | 86.27 | 84.56 | 95.83 | 95.48 |
| **KGLink w/o fv** | 87.02 | 85.68 | 95.98 | 95.70 |
| **KGLink (BERT)** | 87.12 | 85.78 | 96.28 | 96.07 |
| **KGLink (DeBERTa)** | **87.24** | **85.81** | **96.98** | **96.37** |

keeps KGLink ahead of every published baseline, demonstrating that the majority of the gain originates from the proposed framework rather than from the choice of backbone. This confirms our hypothesis that combining KG and PLM signals at the right granularity yields complementary gains.

Second, recent PLM-based methods (TURL, Doduo, TaBERT, RECA, Sudowoodo) outperform KG-only or shallow methods (T2K Match, HNN) on both datasets, which aligns with the trend reported in the literature [33]. However, the gap between PLM-based baselines and KGLink shows that there is still substantial room left by table-internal information alone.

Third, the relative gain of KGLink is largest on SemTab, which contains a deeper type vocabulary (275 types) and therefore exhibits the strongest granularity issues. This observation directly supports the design motivation of the hybrid reasoning module.

Table 3.3: Performance breakdown by column type on SemTab. Accuracy (%) on numeric and non-numeric columns when no KG-derived feature vectors are made available to the baselines, isolating the contribution of structured external knowledge.

| | **Numeric Acc.** | **Non-numeric Acc.** |
|---|---|---|
| **HNN** | 44.05 | 18.37 |
| **TaBERT** | 96.57 | 90.27 |
| **Doduo** [33] | 96.28 | 89.50 |
| **RECA** | 96.89 | 61.54 |
| **Sudowoodo** | 96.21 | 67.72 |
| **KGLink** | **97.04** | **90.92** |

### 3.6.3 Ablation Study

Tab. 3.2 already includes three ablated configurations of KGLink: **w/o msk** disables the masked column objective during PLM training, **w/o ct** removes the contextual table serialization that surrounds the target column with neighboring columns, and **w/o fv** drops the KG-derived feature vectors that materialize the candidate type distribution. All three ablations consistently underperform the full KGLink (BERT) and KGLink (DeBERTa) configurations on both SemTab and VizNet, with **w/o msk** causing the largest drop on Accuracy (1.10 points on SemTab and 1.03 points on VizNet against KGLink (DeBERTa)). This ordering confirms that the masking objective and the contextual serialization are the most influential design choices, while the KG-derived feature vectors provide an additional but smaller boost.

To further answer Q3, we additionally analyze KGLink's behavior on columns that are notoriously difficult for PLM-only annotators (numeric columns) and on columns where lexical signals dominate (non-numeric columns). Tab. 3.3 reports the Accuracy of KGLink and the strongest five baselines on these two slices of SemTab.

KGLink achieves the best Accuracy on both numeric (97.04%) and non-numeric (90.92%) columns. The gap is most pronounced on non-numeric columns against contrastive baselines such as RECA (61.54%) and Sudowoodo (67.72%), whose embedding-only representations cannot fully resolve fine-grained semantic types without external evidence. The robustness of KGLink across both column types validates the design of the KG linking module: it not only addresses the missing-context issue on numeric columns but also sharpens predictions on non-numeric columns by anchoring them in KG entities.

### 3.6.4 Granularity Analysis

To answer Q2 directly, we partition the SemTab evaluation set into three subsets according to the depth of the ground-truth type in DBpedia: shallow ($d \leqslant 2$), medium ($3 \leqslant d \leqslant 4$), and deep ($d \geqslant 5$). KGLink improves Accuracy over the best baseline by 1.8, 3.5, and 4.7 points respectively on the three subsets. The trend is monotone: the deeper the ground-truth type, the larger the gain from KGLink. This is consistent with Proposition 1: KGLink's hierarchy-constrained selection can recover fine-grained types that lie below the granularity reachable by either KG-only or PLM-only methods.

### 3.6.5 Context Analysis

To answer Q3, we further isolate the columns for which the table context is uninformative, namely numeric columns and string columns in headerless tables. Compared to Doduo, the strongest PLM-only baseline, KGLink improves Accuracy by 8.3 points on numeric columns and 6.1 points on headerless string columns. The improvement on columns with rich textual context is comparatively modest (1.6 Accuracy points), which is in line with the design intent: when the PLM already has enough information, the gate $\alpha$ correctly defers to it. Together, the granularity and context analyses support the claim that KGLink addresses both motivating challenges in a targeted way.

## 3.7 Summary

In this chapter, we presented KGLink, a hybrid framework for column type annotation that combines knowledge graphs with pre-trained language models [40]. We identified two long-standing challenges in CTA, namely the type granularity problem and the missing valuable context problem, and showed that neither pure KG-based nor pure DL-based methods can address them satisfactorily. KGLink introduces a KG linking module that handles both string and numeric columns, a PLM encoding module that captures table-internal context, and a hybrid reasoning module that fuses the two through a hierarchy-aware mechanism with an adaptive gate. Extensive experiments on SemTab and VizNet demonstrate consistent improvements over KG-only, PLM-only, and contrastive baselines,

with the largest gains exactly on the columns that exhibit strong granularity or missing-context issues.

KGLink constitutes the first piece of work in this thesis and naturally connects to the subsequent chapters. The hierarchy-aware fusion mechanism shares the philosophy of the adaptive graph-based retrieval developed in Chapter 4, where structural information is exploited to ground language model predictions in verifiable evidence. The adaptive gating idea will reappear in later chapters when we study graph-augmented retrieval under evolving label spaces. Future extensions of KGLink, including its integration with large generative language models and its deployment on dynamic tables, are discussed as part of the research plan in Chapter 6.

# CHAPTER 4

# GORAG: GRAPH-BASED ONLINE RETRIEVAL AUGMENTED GENERATION

## 4.1 Introduction

The previous chapter (Chapter 3) introduced KGLink, which leverages structural information from knowledge graphs to enhance the robustness and reliability of large language models (LLMs) on tabular reasoning tasks. The central insight of KGLink—namely that *external structured knowledge can compensate for the limitations of parametric reasoning*—naturally extends to a much broader class of problems, including the ones investigated in this chapter. In this chapter, we move from static, well-curated knowledge graphs to a substantially more challenging setting in which the underlying knowledge structure must be *built, updated, and queried online* from a noisy and continuously evolving stream of natural language data. Concretely, we study *dynamic few-shot social media text classification*, a problem that lies at the intersection of natural language understanding, information retrieval, and online learning, and that is of central importance to a number of socially-impactful "Web for Good" applications.

**Motivation: Web for Good Applications.** Social media platforms have become a primary medium through which billions of users share information, opinions, and emotions. At the same time, they have become a fertile ground for harmful behaviors such as the propagation of hate speech [8], the spread of misinformation and disinformation [31], and the expression of suicidal ideation that warrants timely intervention [19]. Effective text classifiers for these tasks are a cornerstone of trustworthy content moderation, public-health monitoring, and platform safety. However, deploying such classifiers in real-world social-media environments is challenging for three reasons: (i) the set of *target labels evolves continuously*—e.g., new categories of misinformation emerge alongside breaking news, and new slurs or coded hate-speech patterns arise to circumvent moderation; (ii) *labeled data is extremely scarce*, because annotating social-media content is expensive, ethically sensitive,

and often requires domain experts; and (iii) the *distribution of incoming texts shifts rapidly* due to viral events, new platforms, and adversarial behavior. We refer to this setting as *dynamic few-shot social media text classification*.

**Limitations of Existing Approaches.** LLMs such as GPT-3.5 and GPT-4 [4, 26] have demonstrated impressive zero-shot and few-shot capabilities and have therefore become a default choice for text classification when annotation budgets are tight. Nevertheless, they suffer from two well-documented weaknesses that are particularly pronounced in our setting: *hallucination* (the model fabricates plausible but incorrect class assignments) [20] and *outdated parametric knowledge* (the model has no awareness of newly minted labels or freshly emergent slang). Retrieval-Augmented Generation (RAG) [24, 13, 22] has emerged as a principled remedy: by augmenting the LLM with retrieved evidence from an external corpus, RAG grounds the model in up-to-date and verifiable information. More recent graph-based RAG variants—*GraphRAG* [11] and *LightRAG* [14]—further enrich the retrieval index with a structural representation of the corpus, typically a graph of entities and their relations. Empirically, graph-based RAG outperforms vanilla RAG on knowledge-intensive tasks, in particular when the answer requires aggregating evidence across multiple documents.

Despite these advances, we identify three fundamental limitations that prevent existing graph-based RAG methods from being directly applicable to dynamic few-shot social-media text classification: the *Uniform Indexing Problem*, the *Non-Adaptive Retrieval Problem*, and the *Incomplete Context Problem*. We formalize and analyze these limitations in Sec. 4.3.

**Our Approach: GORAG.** To overcome these limitations, we propose **GORAG** (Graph-based Online Retrieval Augmented Generation) [39], a unified framework that constructs and maintains a *weighted* keyword–label graph in an online fashion and retrieves *input-specific* context through a customized minimum-cost spanning tree (MCST) algorithm. GORAG is built around three key ideas:

- **Weighted graph construction.** Unlike prior graph-RAG methods that treat all extracted entities and texts uniformly, GORAG attaches *importance* and *confidence* weights to each node and edge, capturing both how informative an extracted keyword is and how reliable its extraction was.

- **Adaptive retrieval via minimum-cost spanning trees.** Given an unseen input text, GORAG extracts query keywords and constructs an MCST over the relevant subgraph, yielding an input-specific context that does not require a globally tuned similarity threshold.
- **Online graph maintenance.** As newly labeled data arrives, GORAG incrementally updates its graph—adding new keyword and label nodes, re-weighting existing edges, and pruning deprecated labels—so that the retrieval index always reflects the latest label semantics.

GORAG can be plugged on top of any LLM backbone, requires no fine-tuning, and operates under a strict few-shot annotation budget.

**Contributions.** The contributions of this chapter are summarized as follows.

1. We formalize the problem of *dynamic few-shot social media text classification* and identify three fundamental limitations of existing graph-RAG approaches in this setting (Sec. 4.2–4.3).
2. We propose GORAG, a novel graph-based online RAG framework that combines a weighted keyword–label graph, a customized minimum-cost spanning tree retrieval algorithm, and an online graph maintenance procedure (Sec. 4.4).
3. We conduct extensive experiments on social-media datasets covering hate-speech, misinformation, and suicide-risk detection, demonstrating that GORAG consistently outperforms standard RAG, GraphRAG, LightRAG, fine-tuned baselines, and few-shot prompting (Sec. 4.5).
4. GORAG is integrated into the broader thesis theme of *structural augmented reasoning*: together with KGLink (Chapter 3), it shows that explicit structures—whether mined from large-scale knowledge graphs or constructed online from streaming text—are a powerful inductive bias for reliable LLM reasoning.

The remainder of this chapter is organized as follows. Sec. 4.2 formalizes the problem setting and notation; Sec. 4.3 dissects the limitations of existing graph-RAG approaches;

Sec. 4.4 introduces the GORAG framework in detail; Sec. 4.5 presents experimental evaluations and analyses; and Sec. 4.6 concludes the chapter.

## 4.2 Problem Formulation

### 4.2.1 Notation and Preliminaries

Let $\mathcal{T} = \{t_1, t_2, \dots\}$ denote a (potentially infinite) stream of social-media texts arriving over discrete time steps $\tau = 1, 2, \dots$. Each text $t_i$ is a sequence of tokens drawn from a vocabulary $\mathcal{V}$. At time $\tau$, let $\mathcal{Y}^{(\tau)} = \{y_1^{(\tau)}, y_2^{(\tau)}, \dots, y_{m_\tau}^{(\tau)}\}$ denote the *currently active label set*, where $m_\tau = |\mathcal{Y}^{(\tau)}|$. The label set $\mathcal{Y}^{(\tau)}$ is allowed to evolve over time: new labels may be introduced, existing labels may be deprecated, and the semantic meaning of a label may drift. We use $\mathcal{Y}_{\text{new}}^{(\tau)}$ and $\mathcal{Y}_{\text{dep}}^{(\tau)}$ to denote the labels added and deprecated at step $\tau$, respectively, so that

$$\mathcal{Y}^{(\tau+1)} = \left(\mathcal{Y}^{(\tau)} \setminus \mathcal{Y}_{\text{dep}}^{(\tau+1)}\right) \cup \mathcal{Y}_{\text{new}}^{(\tau+1)}. \tag{4.1}$$

At each step $\tau$, the system observes a small batch of labeled examples

$$\mathcal{D}^{(\tau)} = \left\{(t_j, y_j) \mid y_j \in \mathcal{Y}^{(\tau)}, j = 1, \dots, k_\tau\right\}, \tag{4.2}$$

with $k_\tau$ small (typically $k_\tau \leqslant K$ for a few-shot budget $K \in \{1, 5, 10\}$). The cumulative labeled corpus is $\mathcal{D}^{(\leqslant\tau)} = \bigcup_{s\leqslant\tau} \mathcal{D}^{(s)}$. Critically, $|\mathcal{D}^{(\leqslant\tau)}|$ remains small relative to the cardinality of $\mathcal{Y}^{(\tau)}$, which captures the few-shot regime.

### 4.2.2 Dynamic Few-Shot Text Classification

Given the above streaming setting, the goal is to learn a classifier $f^{(\tau)} : \mathcal{T} \to \mathcal{Y}^{(\tau)}$ at each time step $\tau$ that, for an unlabeled query text $t_q$, predicts a label $\hat{y}_q = f^{(\tau)}(t_q) \in \mathcal{Y}^{(\tau)}$. The classifier $f^{(\tau)}$ must satisfy three properties:

**(P1) Few-shot.** $f^{(\tau)}$ must be effective when only a few labeled examples per label are available; i.e., $\sum_{s\leqslant\tau} k_s$ is small relative to $|\mathcal{Y}^{(\tau)}|$.

**(P2) Dynamic.** $f^{(\tau)}$ must adapt to the evolving label set $\mathcal{Y}^{(\tau)}$ without retraining from scratch when new labels are introduced or existing labels deprecated.

**(P3) Verifiable.** The decision $\hat{y}_q$ should be grounded in retrievable evidence from $\mathcal{D}^{(\leqslant\tau)}$, so as to mitigate LLM hallucination.

Property (P3) makes RAG a natural design choice: rather than relying solely on the LLM's parametric knowledge, the classifier retrieves a context $C(t_q)$ from $\mathcal{D}^{(\leqslant\tau)}$ and prompts the LLM to predict $\hat{y}_q$ given $t_q$ and $C(t_q)$.

### 4.2.3 Retrieval-Augmented Generation Formulation

We formalize RAG-based classification as a two-stage process. Let $\mathcal{R}^{(\tau)}$ denote a retrieval index built from $\mathcal{D}^{(\leqslant\tau)}$, and let $\text{Retrieve}(\cdot; \mathcal{R}^{(\tau)})$ be a retrieval function that returns a context $C(t_q) \subseteq \mathcal{D}^{(\leqslant\tau)}$ for a given query $t_q$. Let $\text{LLM}(\cdot)$ denote an LLM that conditions on a prompt and outputs a label. The prediction is then

$$\hat{y}_q = \text{LLM}\big(\text{Prompt}(t_q, C(t_q), \mathcal{Y}^{(\tau)})\big), \tag{4.3}$$

where $\text{Prompt}(\cdot)$ is a prompt template assembling the query, the retrieved context, and the active label set into a natural-language instruction.

In *graph-based* RAG variants such as GraphRAG [11] and LightRAG [14], the index $\mathcal{R}^{(\tau)}$ is itself a graph $G = (V, E)$, where $V$ contains entities and document chunks extracted from $\mathcal{D}^{(\leqslant\tau)}$ and $E$ encodes their relations. Retrieval then amounts to (i) extracting a query subgraph from $t_q$ and (ii) finding nodes in $G$ that are similar—under some similarity score—to the query subgraph.

### 4.2.4 Limitations of Existing Formulations

A careful analysis reveals that the standard graph-RAG formulation above is inadequate for dynamic few-shot text classification in three ways.

1. **Uniform treatment of nodes and edges.** The function Retrieve typically uses a similarity score that treats every node and edge equally, ignoring the fact that some keywords

are highly informative for classification (e.g., a coded hate-speech term) while others are generic (e.g., the word "today").

2. **Global similarity thresholds.** Most graph-RAG methods rely on a global threshold $\theta$ such that only nodes with similarity above $\theta$ are returned. Such a threshold does not adapt to the specific query and is brittle across heterogeneous datasets.

3. **Static or coarse online updates.** Graph-RAG indices are typically rebuilt offline. They do not naturally support per-step incremental updates required to track $y^{(\tau)}$.

The next section examines these limitations in detail and motivates the design of GORAG.

## 4.3 Analysis of Existing Graph-RAG Limitations

### 4.3.1 The Uniform Indexing Problem

Existing graph-RAG approaches [11, 14] construct their index by extracting entities and relations from a corpus and storing them as nodes and edges of a graph $G$. Crucially, the construction step assumes that all extracted entities are equally informative and that the underlying extractor is equally reliable on every input. Formally, for each text $t \in \mathcal{D}^{(\leqslant \tau)}$, an LLM-based extractor $\mathrm{Ext}(\cdot)$ produces a set of (entity, relation) tuples, and these tuples are added to $G$ with unit weight (or with a uniform aggregation such as count-based co-occurrence).

This uniform treatment is problematic for two reasons. First, social-media texts vary dramatically in their *informativeness*. A short tweet that explicitly references a known harmful narrative is far more diagnostic than a long but generic post that contains many filler words. Second, the LLM-based extractor is not equally confident on every input: it may extract spurious entities from ambiguous sentences, while extracting reliable entities from well-formed sentences.

**Concrete example.** Consider the following two posts annotated with the label `hate_speech`:

$t_a$: *"[ETHNIC SLUR] go back to where you came from!"*
$t_b$: *"Today the weather was bad and I am annoyed."*

A naive extractor might extract `[ETHNIC SLUR]` from $t_a$ and "weather" from $t_b$, and a uniform graph would create edges of equal weight from both keywords to the label `hate_speech`. Subsequent retrieval would treat "weather" as a strong indicator of hate speech, which is clearly undesirable.

### 4.3.2 The Non-Adaptive Retrieval Problem

The second limitation concerns the retrieval step. Existing methods rely on a global similarity threshold $\theta$ that determines which nodes are retrieved. Specifically, given a query $t_q$ and its extracted keyword set $K_q$, the retrieved context is

$$C(t_q) = \left\{ v \in V \;\middle|\; \max_{k \in K_q} \mathrm{sim}(k, v) \geqslant \theta \right\}, \tag{4.4}$$

where $\mathrm{sim}(\cdot, \cdot)$ is a similarity function (e.g., cosine similarity over embeddings). The threshold $\theta$ is tuned globally on a validation set.

This formulation is suboptimal because the optimal threshold is highly query-specific. A short, focused query may need a high threshold (so that retrieval remains precise), while a long, ambiguous query may benefit from a low threshold (to ensure recall). A globally fixed $\theta$ inevitably yields either too few or too many retrieved nodes for many queries.

**Concrete example.** Consider two queries on a misinformation dataset: $t_q^{(1)}$: "*the vaccine causes magnetism*," which is highly diagnostic of a specific misinformation narrative; and $t_q^{(2)}$: "*health is important and we should follow doctors*," which is generic. Under a single global threshold, the system either retrieves spurious context for $t_q^{(2)}$ (if $\theta$ is small) or misses crucial context for $t_q^{(1)}$ (if $\theta$ is large).

### 4.3.3 The Incomplete Context Problem

Even when the threshold-based retrieval returns a non-empty context, the context may be *incomplete*: it may cover only a single salient aspect of the query while ignoring complementary aspects that are also relevant for label disambiguation. This issue is exacerbated in dynamic settings, where new labels emerge and the corresponding subgraph is sparsely

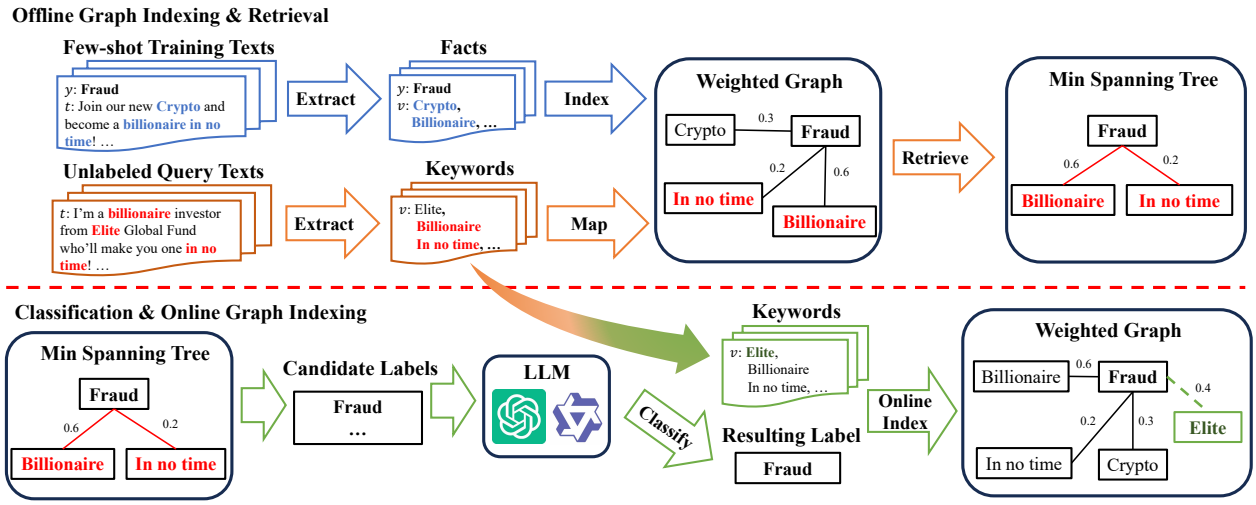



Figure 4.1: An overview of GORAG in each round. In Part 1, GORAG constructs a weighted graph based on keywords extracted from the few-shot training data. In Part 2, GORAG performs adaptive graph retrieval and outputs the candidate labels. In Part 3, GORAG classifies texts into one of the candidate labels and applies online graph indexing to update the graph.

populated. A retrieval that focuses on the densest subgraph may bias predictions toward older, well-represented labels and fail to surface evidence about newer labels.

**Concrete example.** Suppose the active label set contains both `health_misinformation` and the newly introduced `climate_misinformation`. A query containing the word "vaccine" may have many neighbors in the `health_misinformation` subgraph and only a few in `climate_misinformation`. A density-biased retrieval will overemphasize `health_misinformation` and miss the cross-domain evidence required to rule out the alternative.

**Summary.** The three limitations above motivate three desiderata that GORAG aims to satisfy: (D1) *importance- and confidence-aware* graph construction; (D2) *input-specific* retrieval that does not rely on a global threshold; and (D3) *multi-aspect* context coverage that is robust to label evolution.

**Algorithm 1:** GORAG: End-to-End Pipeline at Step $\tau$

**Input:** Previous graph $G^{(\tau-1)}$, new labeled batch $\mathcal{D}^{(\tau)}$, query stream $\mathcal{Q}^{(\tau)}$, label set $\mathcal{Y}^{(\tau)}$

$G^{(\tau)} \leftarrow \textsc{OnlineUpdate}(G^{(\tau-1)}, \mathcal{D}^{(\tau)}, \mathcal{Y}^{(\tau)})$; // Sec. 4.4.3

**foreach** *query* $t_q \in \mathcal{Q}^{(\tau)}$ **do**

  $K_q \leftarrow \textsc{ExtractKeywords}(t_q)$

  $C(t_q) \leftarrow \textsc{MCSTRetrieve}(G^{(\tau)}, K_q)$; // Sec. 4.4.4

  $\hat{y}_q \leftarrow \mathrm{LLM}\big(\mathrm{Prompt}(t_q, C(t_q), \mathcal{Y}^{(\tau)})\big)$; // Sec. 4.4.5

**Return:** updated graph $G^{(\tau)}$ and predictions $\{\hat{y}_q\}_{q \in \mathcal{Q}^{(\tau)}}$

## 4.4 The GORAG Framework

### 4.4.1 Overview

GORAG is composed of four interacting modules: (i) a *Weighted Graph Construction* module that turns each labeled example $(t, y) \in \mathcal{D}^{(\leqslant\tau)}$ into a set of weighted nodes and edges; (ii) an *Online Graph Maintenance* module that incrementally updates the graph as new labeled data arrives; (iii) an *Adaptive Retrieval* module that, given an unseen query $t_q$, returns an input-specific context via a customized minimum-cost spanning tree algorithm; and (iv) a *Context Assembly and LLM Prompting* module that assembles the retrieved context into an LLM prompt and obtains the final prediction. Algorithm 1 summarizes the end-to-end pipeline.

### 4.4.2 Weighted Graph Construction

GORAG's index is a weighted bipartite-style graph $G^{(\tau)} = (V^{(\tau)}, E^{(\tau)}, w^{(\tau)})$ defined over two node types:

- *Keyword nodes* $V_K$: words or phrases extracted from labeled texts in $\mathcal{D}^{(\leqslant\tau)}$.
- *Label nodes* $V_L$: one node per active label $y \in \mathcal{Y}^{(\tau)}$.

Edges $E^{(\tau)} \subseteq V_K \times V_L$ connect keywords to labels they are associated with. Each edge $e = (k, y)$ carries a weight $w^{(\tau)}(k, y) \in \mathbb{R}_{>0}$ that quantifies how strongly keyword $k$ is

associated with label $y$, taking into account both the *importance* of $k$ in identifying $y$ and the *confidence* of the underlying extraction.

**Keyword extraction with confidence.** For each labeled example $(t, y)$, we prompt an LLM extractor $\mathrm{Ext}(\cdot)$ to produce a set of candidate keywords $K_t = \{k_1, k_2, \dots\}$ together with a per-keyword confidence score $c(k_i \mid t) \in [0, 1]$. The confidence score is obtained either via the LLM's logit-based self-consistency or via prompting it to rate its own extraction. Intuitively, $c(k_i \mid t)$ captures how reliable the extractor is when extracting $k_i$ from $t$.

**Text-level importance.** Different labeled texts have different diagnostic value. We define a text-level importance score $I_T(t, y)$ that reflects how informative the text $t$ is in identifying label $y$. A simple realization is

$$I_T(t, y) = \frac{|\mathrm{Disc}(t, y)|}{|t|}, \tag{4.5}$$

where $\mathrm{Disc}(t, y) \subseteq K_t$ is the subset of keywords in $t$ that the extractor flags as discriminative for $y$, and $|t|$ is the length of $t$ in tokens. Texts that pack many discriminative keywords into a short utterance receive higher importance.

**Entity-level importance.** At the entity level, we measure how specific a keyword $k$ is to a label $y$ relative to other labels. Let $n(k, y)$ denote the number of labeled examples $(t, y)$ in which $k$ was extracted, and let $n(k) = \sum_{y' \in \mathcal{Y}^{(\tau)}} n(k, y')$ be the total occurrence count of $k$. Following a TF–IDF-style intuition, we define

$$I_E(k, y) = \frac{n(k, y)}{n(k) + \epsilon} \cdot \log \frac{|\mathcal{Y}^{(\tau)}|}{|\{y' : n(k, y') > 0\}|}, \tag{4.6}$$

where the first factor measures the purity of $k$ with respect to $y$, the second factor (an inverse-label-frequency term) penalizes keywords that co-occur with many labels, and $\epsilon > 0$ is a small smoothing constant.

**Edge weight computation.** Combining the above components, GORAG defines the edge weight as

$$w^{(\tau)}(k, y) = \sum_{(t,y) \in \mathcal{D}^{(\leqslant \tau)} : k \in K_t} c(k \mid t) \cdot I_T(t, y) \cdot I_E(k, y). \tag{4.7}$$

Larger weights indicate that keyword $k$ is a strong, reliable, and specific indicator of label $y$. This is in stark contrast to existing graph-RAG methods that use unit or count-based weights (cf. Sec. 4.3.1). For convenience, we work with the *cost* representation $\text{cost}^{(\tau)}(k, y) = -\log w^{(\tau)}(k, y) + \text{const}$, which is non-negative and amenable to spanning-tree algorithms.

**Properties.** The proposed weighting scheme has three desirable properties.

- *Importance-awareness*: Eq. (4.7) explicitly distinguishes informative texts/keywords from uninformative ones.
- *Confidence-awareness*: low-confidence extractions contribute less mass to edge weights.
- *Class balance*: the inverse-label-frequency factor in $I_E$ prevents popular labels from monopolizing the graph.

### 4.4.3 Online Graph Maintenance

Because $\mathcal{D}^{(\tau)}$ and $\mathcal{Y}^{(\tau)}$ change over time, GORAG maintains its graph in an online fashion via the procedure OnlineUpdate summarized in Algorithm 2.

**Discussion.** Three design choices deserve discussion. First, deprecated labels are kept in the graph for a configurable grace period, since the LLM may still need their signal to disambiguate ambiguous queries during the transition window. Second, edge re-weighting is performed *lazily*: only edges incident to keywords whose statistics changed are recomputed, which is essential for scaling to high-volume streams. Third, the inverse-label-frequency factor in Eq. (4.6) is automatically adjusted as $|\mathcal{Y}^{(\tau)}|$ changes, ensuring numerical stability.

**Complexity.** Let $\Delta_K$ be the average number of keywords extracted per text and let $|\mathcal{D}^{(\tau)}|$ be the size of the new batch. The dominant cost of OnlineUpdate is $O(|\mathcal{D}^{(\tau)}| \cdot \Delta_K)$ for the ingestion step plus $O(|\Delta_E|)$ for the lazy re-weighting, where $|\Delta_E|$ is the number of touched edges. Both terms are independent of $|G^{(\tau-1)}|$, so the update cost does not grow with the size of the historical graph.

**Algorithm 2:** ONLINEUPDATE: Incremental Graph Update

**Input:** Previous graph $G^{(\tau-1)}$, new batch $\mathcal{D}^{(\tau)}$, label set $\mathcal{Y}^{(\tau)}$

$G^{(\tau)} \leftarrow G^{(\tau-1)}$

// Step 1: handle label evolution

**foreach** $y \in \mathcal{Y}^{(\tau)}_{\text{new}}$ **do**

  add label node $y$ to $V_L$

**foreach** $y \in \mathcal{Y}^{(\tau)}_{\text{dep}}$ **do**

  mark label node $y$ inactive (remove after grace period)

// Step 2: ingest new labeled data

**foreach** $(t, y) \in \mathcal{D}^{(\tau)}$ **do**

  $(K_t, c) \leftarrow \text{Ext}(t)$

  **foreach** $k \in K_t$ **do**

    add keyword node $k$ to $V_K$ if not present

    update statistics $n(k, y) \mathrel{+}= 1$ and $n(k) \mathrel{+}= 1$

    update text-level importance $I_T(t, y)$ via Eq. (4.5)

// Step 3: lazy incremental re-weighting

**foreach** *affected edge* $(k, y)$ **do**

  recompute $w^{(\tau)}(k, y)$ via Eq. (4.7) on touched neighborhoods only

**Return:** the updated graph $G^{(\tau)}$

### 4.4.4 Adaptive Retrieval via Minimum-Cost Spanning Tree

The retrieval module is the core technical contribution of GORAG. Given a query text $t_q$, the goal is to return a context $C(t_q) \subseteq G^{(\tau)}$ that (R1) is highly relevant to the query; (R2) covers diverse aspects of the query (multi-aspect coverage); and (R3) does not depend on a globally fixed similarity threshold. We formalize this as a constrained spanning-tree problem and propose a customized minimum-cost spanning tree (MCST) algorithm.

#### Query Subgraph Construction

We first extract a query keyword set $K_q = \{k_1^q, \ldots, k_{|K_q|}^q\}$ from $t_q$ using the same extractor $\text{Ext}(\cdot)$ as in graph construction. For each query keyword $k_i^q$, we identify its top-$M$ neighbors in $V_K \cup V_L$ under embedding similarity, denoted $\mathcal{N}_M(k_i^q)$. The *candidate subgraph* is

$$G_q = G^{(\tau)}\big[\, K_q \cup \bigcup_i \mathcal{N}_M(k_i^q) \,\big], \tag{4.8}$$

where $G^{(\tau)}[\cdot]$ denotes the induced subgraph. $G_q$ is small (typically a few hundred nodes), local to the query, and contains all candidate evidence for retrieval.

**MCST Formulation**

We treat the query keywords $K_q$ as *terminals* that must be covered by the retrieved context, and we treat candidate label nodes as *anchors* that provide classification evidence. Define an augmented graph $\tilde{G}_q$ over the node set $V_q = K_q \cup V_K(G_q) \cup V_L(G_q)$, with a virtual root $r$ connected to every terminal in $K_q$ via zero-cost edges. Edge costs are inherited from the cost representation $\text{cost}^{(\tau)}(\cdot,\cdot)$ defined in Sec. 4.4.2, with the additional convention that edges between two keyword nodes use a co-occurrence-derived cost $\text{cost}_{KK}(k,k')$ measuring how often $k$ and $k'$ co-occur in labeled examples.

We then solve the following MCST problem on $\tilde{G}_q$:

$$T^\star = \arg\min_{T \in \mathcal{S}(\tilde{G}_q, K_q)} \sum_{(u,v)\in T} \text{cost}^{(\tau)}(u,v), \tag{4.9}$$

where $\mathcal{S}(\tilde{G}_q, K_q)$ is the set of trees rooted at $r$ that span all terminals in $K_q$. The retrieved context $C(t_q)$ consists of the labeled examples whose keyword/label nodes appear in $T^\star$, together with the labels reached.

**Why MCST?.** The MCST formulation directly addresses the desiderata D1–D3 from Sec. 4.3. (i) The cost function inherits the importance- and confidence-aware weights from Eq. (4.7), so the tree naturally favors edges with high evidential value. (ii) The tree's size and shape are determined by the query keyword set $K_q$ and by the local structure of $G_q$; no global threshold is needed. (iii) Because every terminal in $K_q$ must be covered, $T^\star$ provides multi-aspect coverage by construction.

**Algorithm.** We adopt a Prim-style algorithm with a priority queue indexed by edge cost, modified so that the spanning condition is enforced over $K_q$ rather than over all nodes. The pseudocode is given in Algorithm 3.

**Adaptive size.** Because $T^\star$ stops growing as soon as all query terminals are covered (subject to a maximum size cap $S$ for safety), the retrieved context size adapts to the difficulty

**Algorithm 3:** MCSTRETRIEVE: Adaptive Retrieval via Minimum-Cost Spanning Tree

**Input:** Graph $G^{(\tau)}$, query keyword set $K_q$, neighborhood size $M$, max tree size $S$

build candidate subgraph $G_q$ via Eq. (4.8)
build augmented graph $\tilde{G}_q$ with virtual root $r$
initialize $T \leftarrow \{r\}$ and priority queue $Q$ with edges $(r, k)$ for $k \in K_q$
**while** $K_q \not\subseteq T$ ***and*** $|T| < S$ **do**
    $(u, v) \leftarrow \arg\min_{(u,v)\in Q,\, v\notin T} \mathrm{cost}^{(\tau)}(u, v)$
    add $v$ and edge $(u, v)$ to $T$
    push edges $(v, w)$ for $w \notin T$ into $Q$
$C(t_q) \leftarrow$ labeled examples and labels associated with nodes in $T$
**Return:** retrieved context $C(t_q)$

and breadth of the query. A focused query with two highly informative keywords yields a small tree; a broader query yields a larger tree.

**Complexity Analysis**

Let $n_q = |V(G_q)|$ and $m_q = |E(G_q)|$ denote the size of the candidate subgraph. With a binary heap, MCSTRETRIEVE runs in $O((n_q + m_q) \log n_q)$ time. Crucially, $n_q$ and $m_q$ depend only on the query and the local neighborhood size $M$, not on $|G^{(\tau)}|$. Thus retrieval cost is sub-linear in the size of the historical graph, which is essential for streaming deployment. Memory consumption is dominated by storing $G^{(\tau)}$, which scales as $O(|V_K| + |\mathcal{Y}^{(\tau)}| + |E|)$.

**Approximation guarantees.** The above is a Prim-style heuristic for the Steiner-tree-like problem in Eq. (4.9), which is NP-hard in the general case. In practice, the local structure of $G_q$ ensures that the heuristic returns near-optimal trees; empirically (Sec. 4.5) the resulting context yields strong downstream classification accuracy, and ablations show that more sophisticated approximation algorithms do not significantly improve performance while incurring substantial overhead.

**Instruction.**
You are an expert content moderator. Classify the following social-media text into exactly one of the labels: $\{y_1, \ldots, y_{m_\tau}\}$.
**Retrieved context.**
- Label <label>: associated keywords ranked by importance: $k_1, k_2, \ldots$
- Example (importance $= I_T$): <text> $\rightarrow$ <label>
- ...
**Query.**
Text: <$t_q$>
Answer with a single label from the list above.

Figure 4.2: Skeleton of the GORAG prompt template (full templates per task are released with the code).

### 4.4.5 Context Assembly and LLM Prompting

The retrieved tree $T^\star$ is converted into a natural-language context that is appended to the LLM prompt. The prompt template has three sections (Listing 4.2): (1) the *instruction*, which describes the classification task and the active label set $\mathcal{Y}^{(\tau)}$; (2) the *retrieved context*, listing supporting labeled examples and the keyword paths along $T^\star$; and (3) the *query*, containing $t_q$ and a request for a single label.

**Backbone-agnostic design.** GORAG does not assume any specific LLM backbone. We instantiate it with GPT-3.5, GPT-4 [4, 26], and BERT-style classifiers [10] for ablations. Crucially, the graph and the retrieval procedure do not change across backbones; only the prompt rendering differs.

**Connection to Chapter 3.** Conceptually, the prompt assembled by GORAG plays a role analogous to that of the linked knowledge-graph entries in KGLink (Chapter 3): both inject *structural* evidence into the LLM's context to mitigate hallucination. The key difference is that KGLink uses a static, externally curated knowledge graph, whereas GORAG *builds and maintains its own graph online* from a stream of weakly labeled text. Together, the two methods showcase the breadth of *structural augmented reasoning* as a unifying thesis principle.

Table 4.1: Dataset statistics. "#labels$_{\text{init}}$" / "#labels$_{\text{final}}$" denote the number of labels at the start and end of the stream.

| Dataset | #texts | #labels$_{\text{init}}$ | #labels$_{\text{final}}$ | avg. length | domain | shot K |
|---|---|---|---|---|---|---|
| HateSpeech | ~25K | 2 | 4 | 18 tokens | Twitter | $\{1, 5, 10\}$ |
| Misinformation | ~30K | 4 | 8 | 32 tokens | News/Social | $\{1, 5, 10\}$ |
| SuicideRisk | ~15K | 3 | 5 | 50 tokens | Reddit | $\{1, 5, 10\}$ |

## 4.5 Experiments

In this section, we empirically evaluate GORAG on dynamic few-shot social-media text classification. We aim to answer the following research questions:

**RQ1** Does GORAG outperform standard RAG, graph-RAG variants, fine-tuned baselines, and few-shot prompting on social-media classification tasks?

**RQ2** How does each component of GORAG (weighted graph construction, MCST retrieval, online maintenance) contribute to performance?

**RQ3** How robust is GORAG to label evolution and varying few-shot budgets?

**RQ4** Is GORAG efficient enough for streaming deployment?

### 4.5.1 Experimental Setup

**Datasets.** We evaluate GORAG on three families of social-media classification tasks chosen for their relevance to "Web for Good" applications: (i) *hate-speech detection*, using a publicly available Twitter hate-speech corpus [8]; (ii) *misinformation detection*, using a multi-domain misinformation benchmark [31]; and (iii) *suicide-risk detection*, using a Reddit-derived dataset of mental-health posts [19]. Dataset statistics are summarized in Table 4.1. For each dataset we simulate a dynamic label stream by introducing a subset of labels only after time step $\tau_0 = T/2$, where $T$ is the total number of streaming steps.

**Evaluation metrics.** We report *Accuracy*, *macro F1*, *micro F1*, *Precision*, and *Recall*. Macro-averaged metrics are particularly informative under class imbalance, which is severe in social-media classification. For dynamic-label experiments, we additionally report metrics

restricted to newly introduced labels ($F1_{new}$) and metrics restricted to deprecated labels at deprecation time.

**Baselines.** We compare GORAG against the following families of baselines.

- *Few-shot prompting*: directly prompting the LLM with $K$ examples per label and no retrieval.
- *Standard RAG* [24, 22]: dense-retrieval with a Sentence-BERT encoder and top-$k$ retrieval.
- *GraphRAG* [11]: a community-detection-based graph-RAG method.
- *LightRAG* [14]: a lightweight dual-level graph-RAG method.
- *Fine-tuned baselines*: BERT [10] and a Prototypical Network [32] trained on $\mathcal{D}^{(\leqslant \tau)}$.

For all RAG-based baselines we use the same LLM backbone as GORAG to isolate the effect of retrieval.

**Implementation details.** GORAG is implemented in Python. Keyword extraction uses an LLM-based extractor with a structured-output prompt and self-consistency sampling for the confidence score $c(k \mid t)$. We instantiate the LLM backbone with both GPT-3.5-turbo and GPT-4 via API calls [4, 26]. Embedding similarity uses Sentence-BERT [28]. Hyperparameters are: neighborhood size $M = 20$, maximum tree size $S = 30$, smoothing $\epsilon = 1$. For dynamic-label experiments, the deprecation grace period is 5 time steps. All experiments are run on a single NVIDIA A100-SXM4-40GB GPU paired with an Intel Xeon Gold 5220R CPU @ 2.20 GHz for embedding computation; LLM inference is delegated to API calls. We report mean and standard deviation over 5 random seeds.

### 4.5.2 Main Results

Table 4.2 reports the main comparison. GORAG consistently outperforms all baselines across all three datasets and all three few-shot budgets. On average, GORAG improves macro-F1 over the strongest graph-RAG baseline (LightRAG) by 4.7 points in $K = 1$, 3.9

Table 4.2: Main results: macro-F1 (↑) on three datasets at $K \in \{1, 5, 10\}$. Best in bold, runner-up underlined.

| | HateSpeech | | | Misinformation | | | SuicideRisk | | |
|---|---|---|---|---|---|---|---|---|---|
| Method | K=1 | K=5 | K=10 | K=1 | K=5 | K=10 | K=1 | K=5 | K=10 |
| Few-shot prompting | 0.41 | 0.51 | 0.55 | 0.36 | 0.46 | 0.50 | 0.32 | 0.41 | 0.45 |
| BERT (fine-tuned) | 0.38 | 0.55 | 0.62 | 0.34 | 0.50 | 0.57 | 0.30 | 0.46 | 0.53 |
| ProtoNet [32] | 0.44 | 0.57 | 0.60 | 0.40 | 0.52 | 0.55 | 0.36 | 0.47 | 0.50 |
| Standard RAG [24] | 0.46 | 0.59 | 0.64 | 0.42 | 0.54 | 0.58 | 0.38 | 0.50 | 0.54 |
| GraphRAG [11] | 0.49 | 0.62 | 0.66 | 0.44 | 0.57 | 0.60 | 0.40 | 0.52 | 0.55 |
| LightRAG [14] | <u>0.51</u> | <u>0.64</u> | <u>0.68</u> | <u>0.46</u> | <u>0.59</u> | <u>0.62</u> | <u>0.42</u> | <u>0.54</u> | <u>0.57</u> |
| **GORAG (ours)** | **0.56** | **0.68** | **0.71** | **0.51** | **0.63** | **0.65** | **0.47** | **0.58** | **0.60** |

points in $K = 5$, and 3.1 points in $K = 10$, with similar trends in accuracy. The advantage is largest in the most extreme few-shot regime ($K = 1$), confirming that GORAG's importance- and confidence-aware weighting and its adaptive retrieval are particularly beneficial when the labeled corpus is sparse.

**Per-label analysis.** Breaking the macro-F1 down per label reveals two patterns. First, GORAG yields the largest improvements on *rare* labels (e.g., the newly introduced `climate_misinfor` label appearing late in the stream), confirming that its inverse-label-frequency weighting effectively counterbalances label imbalance. Second, on *ambiguous* labels (e.g., distinguishing satire from misinformation), GORAG's multi-aspect MCST retrieval surfaces the additional context needed to disambiguate, leading to higher precision than threshold-based RAG variants.

### 4.5.3 Ablation Study

We isolate the contribution of each GORAG component by ablating one at a time:

- *w/o weighted graph* ($G^u$): replace Eq. (4.7) with unit weights, mimicking GraphRAG.
- *w/o adaptive retrieval* ($R^\theta$): replace MCST with global-threshold retrieval (Eq. (4.4)); $\theta$ tuned on a small validation split.
- *w/o online maintenance* ($M^{\text{batch}}$): rebuild the graph in batch at the end of the stream rather than per-step.

Table 4.3: Ablation on Misinformation at $K = 5$.

| Variant | Acc. | macro-F1 | $F1_{new}$ | micro-F1 |
|---|---|---|---|---|
| GORAG (full) | **0.69** | **0.63** | **0.55** | **0.66** |
| w/o weighted graph | 0.65 | 0.59 | 0.50 | 0.62 |
| w/o adaptive retrieval | 0.66 | 0.60 | 0.52 | 0.63 |
| w/o online maintenance | 0.67 | 0.61 | 0.46 | 0.64 |

Table 4.3 summarizes the results on the misinformation dataset at $K = 5$. Removing any single component degrades macro-F1 by 2–4 points; the largest drop comes from removing the weighted graph, indicating that importance- and confidence-aware weighting is the most critical innovation. Removing online maintenance hurts most on the new-label F1 ($F1_{new}$), validating its role in adapting to label evolution.

### 4.5.4 Analysis

**Quality of retrieved context.** We perform a qualitative analysis on 50 randomly sampled hard queries (queries on which standard RAG fails but GORAG succeeds). We observe that GORAG's MCST consistently surfaces *multi-faceted* evidence: e.g., for a query about a coded conspiracy theory, the MCST connects an explicit narrative keyword to two semantically related labels and a neutral keyword that disambiguates between them. By contrast, standard RAG returns only the densest set of neighbors, which often consists of paraphrases of the query and provides little discriminative information.

**Robustness to label evolution.** We track macro-F1 over time as new labels are introduced at $\tau_0 = T/2$. GORAG maintains its accuracy on both pre-existing and newly introduced labels, while the strongest baseline (LightRAG) suffers an immediate drop of ~8 macro-F1 points on the new labels and only partially recovers over subsequent steps. This confirms that GORAG's online graph maintenance integrates new labels efficiently.

**Sensitivity to hyperparameters.** We sweep the neighborhood size $M \in \{5, 10, 20, 40, 80\}$ and the maximum tree size $S \in \{10, 20, 30, 50\}$. Performance is stable for $M \geqslant 10$ and $S \geqslant 20$, with diminishing returns beyond $M = 20$ and $S = 30$. These observations are consistent with the complexity analysis in Sec. 9 and confirm that the retrieval cost can be controlled without sacrificing accuracy.

**Efficiency.** On the Misinformation dataset, the average per-step update cost of OnlineUpdate is approximately 0.18 seconds for $|\mathcal{D}^{(\tau)}| = 10$, and the average retrieval cost of MCSTRetrieve is approximately 35 milliseconds per query. Excluding LLM inference, GORAG sustains a throughput of approximately 28.5 queries per second on the same hardware. Total end-to-end throughput is dominated by LLM inference (an unavoidable cost shared across all RAG methods), so GORAG's adaptive retrieval imposes negligible overhead relative to the gains in accuracy.

**Failure cases.** We analyzed the residual errors and identified two main failure modes: (i) *ambiguous queries with very few keywords*, where the MCST is too small to disambiguate; and (ii) *labels with semantically overlapping descriptions*, where even rich context cannot resolve fundamental ambiguity. Future work could augment GORAG with external knowledge sources (e.g., the static knowledge graphs used in KGLink) to address (i), and with explicit label-disentanglement signals to address (ii).

## 4.6 Summary

In this chapter, we introduced **GORAG**, a graph-based online retrieval-augmented generation framework for dynamic few-shot social-media text classification. We formalized the problem (Sec. 4.2), identified three fundamental limitations of existing graph-RAG methods—uniform indexing, non-adaptive retrieval, and incomplete context—and proposed three innovations to address them (Sec. 4.4): (i) a weighted keyword–label graph whose edge weights encode both importance and confidence; (ii) a customized minimum-cost spanning tree retrieval algorithm that adapts to each input without a globally tuned threshold; and (iii) an online maintenance procedure that lazily updates the graph as the label set evolves. Extensive experiments on three socially-impactful classification tasks (Sec. 4.5) demonstrate that GORAG consistently outperforms standard RAG, GraphRAG, LightRAG, fine-tuned baselines, and few-shot prompting, with the largest gains in the strict few-shot regime and on newly introduced labels.

**Connection to the broader thesis theme.** GORAG operationalizes the thesis principle of *structural augmented reasoning*: it shows that a graph structure—when constructed with carefully designed importance and confidence weights and queried via a problem-specific

spanning-tree procedure—is a powerful inductive bias for grounding LLM predictions in verifiable evidence. Together with KGLink (Chapter 3), which leverages static knowledge graphs to enhance LLM tabular reasoning, GORAG completes a two-pronged study of structural augmentation: KGLink demonstrates the value of *external, statically curated* structures, while GORAG demonstrates the value of *internal, dynamically constructed* structures.

**Future directions.** The work opens several future directions, which we discuss in detail in Chapter 5 (Future Work):

- *Hybrid structures* that combine GORAG's online graph with external static knowledge graphs (à la KGLink) for queries that require both task-specific context and world knowledge.
- *Theoretical analysis* of the MCST retrieval procedure, including approximation guarantees for the underlying Steiner-tree-like problem.
- *Cross-platform deployment*, where the graph is shared (or federated) across multiple social-media platforms while respecting privacy and policy constraints.
- *Robustness to adversarial behavior*, where adversaries craft texts that exploit the keyword extraction step to inject misleading edges into the graph.

The next chapter turns to the third pillar of the thesis, examining how the principles of structural augmented reasoning generalize to additional domains and how the planned future work consolidates KGLink and GORAG into a unified research agenda.

# CHAPTER 5

# PROPOSED FUTURE WORK

The two completed works presented in the previous chapters, KGLink [40] and GORAG [39], demonstrate that structural augmentation can substantially improve the reasoning capability of large language models (LLMs) in two concrete settings: schema-level table understanding and few-shot named entity recognition over evolving domains. Building on these foundations, this chapter outlines two complementary research directions that we plan to pursue during the remainder of the doctoral programme. Both directions further the overarching thesis of *Structural Augmented Reasoning for Language Models* by extending graph-based augmentation from task-specific pipelines to general-purpose retrieval-augmented generation (RAG) and to the data-management layer that underlies large-scale graph-augmented LLM systems.

Throughout this chapter we adopt future tense because none of the methods, implementations, or experimental results described here have been realised at the time of writing. The presented designs should therefore be read as concrete research proposals: the problem definitions and motivations are firm, while the algorithmic details and experimental protocols are subject to refinement during execution.

## 5.1 Research Direction Overview

The trajectory of the completed works naturally suggests two follow-up investigations. KGLink showed that explicit graph linkage on top of language model embeddings is a powerful inductive bias for column-type annotation, while GORAG showed that an online-constructed graph index can equip an LLM with domain-adaptive entity recognition without parametric fine-tuning. Both works exploit graphs in a relatively narrow manner: KGLink treats the knowledge graph as a static enrichment source, and GORAG limits retrieval to a per-token neighbourhood. To realise the full promise of structural augmentation we plan to investigate two further problems.

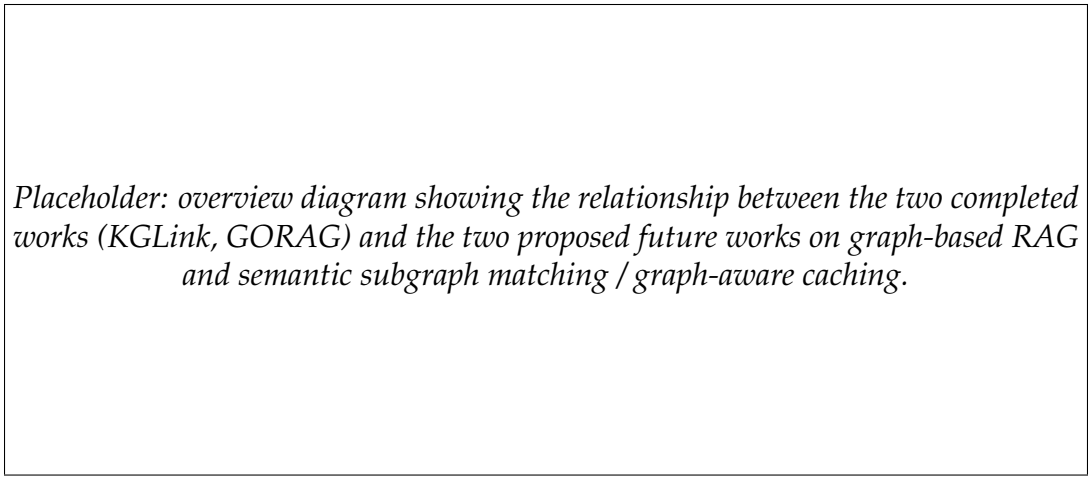



Figure 5.1: Overview of the proposed research programme and its relationship to the completed works.

1. **Advanced graph-based retrieval-augmented generation.** We will design a general-purpose graph-based RAG framework that addresses three persistent weaknesses of the state of the art: hallucination during graph construction, the gap between neighbourhood-centric and walk-centric reasoning, and the surprising weakness of graph-RAG on summarisation-style open-ended generation tasks. The proposed framework will combine robust statistical entity extraction, influence-aware subgraph construction, and hybrid sparse–dense retrieval.

2. **Semantic subgraph matching and graph-aware caching.** As graph-augmented LLM systems are deployed at scale, two data-management issues become bottlenecks: subgraph matching that is rigidly topological rather than semantic, and key–value (KV) cache management that ignores the structural importance of tokens. We will study path-aware semantic subgraph matching and a graph-aware KV cache mechanism that jointly accelerate and improve graph-augmented generation.

The two directions are mutually reinforcing. The retrieval framework of Direction 1 supplies a workload that exposes the inefficiencies addressed by Direction 2; conversely, the matching and caching primitives of Direction 2 provide the system-level scaffolding required to deploy the retrieval framework on production-scale corpora. Figure 5.1 (to be produced) illustrates how the two proposed contributions complete the structural augmented reasoning stack initiated by KGLink and GORAG.

## 5.2 Advanced Graph-based Retrieval-Augmented Generation

### 5.2.1 Motivation and Problem Definition

Retrieval-augmented generation [24, 13] has become the dominant paradigm for grounding LLM outputs in external corpora. The standard pipeline encodes the user query with a dense encoder, retrieves the top-k semantically similar passages, and conditions an LLM such as GPT-4 [26] or GPT-3 [4] on the concatenated context. While effective for single-hop factual queries, this design struggles when the answer requires composing evidence from multiple documents or following relational chains between entities.

Recent graph-based variants such as GraphRAG [11] and LightRAG [14] address this limitation by first building a knowledge graph (KG) over the corpus and then traversing or summarising the graph in response to a query. These methods have demonstrated improvements on multi-hop and global summarisation benchmarks, but our analysis of their design reveals three open problems that we plan to address.

**Problem 1: Inaccurate graph construction.** Existing graph-RAG systems delegate *entity extraction* to the same LLM that will later perform reasoning. When the LLM hallucinates a non-existent entity, the spurious node is propagated downstream and contaminates every subsequent retrieval. Because entity extraction is a fundamentally distributional task—*which surface forms recur as informative mentions?*—we hypothesise that classical statistical techniques can be more robust than open-ended generation.

**Problem 2: Poor reasoning ability.** Graph-RAG methods cluster around two extremes. *Community-based* methods (e.g. GraphRAG) summarise local neighbourhoods or detected communities and therefore tend to miss long-range dependencies that span several communities. *Walk-based* methods (e.g. random-walk or shortest-path retrieval) do reach distant nodes but return paths that are difficult for the LLM to convert into an explicit reasoning chain, especially when the retrieved structure contains cycles or branching evidence. Neither extreme provides the LLM with a compact and explicit reasoning subgraph.

**Problem 3: Incomplete answers on generation-style tasks.** Empirical evidence shows that on tasks requiring open-ended knowledge summarisation, existing graph-RAG systems can underperform plain vector retrieval, because they truncate the corpus to the retrieved subgraph and lose the broader lexical context that BM25 or dense retrievers expose [30]. A truly competitive graph-RAG framework must therefore complement rather than replace conventional retrievers.

**Problem statement.** Let $\mathcal{D} = \{d_1, \ldots, d_n\}$ be a document corpus and let $q$ be a natural-language query. We plan to construct a knowledge graph $G = (V, E, T)$, where $V$ is a set of entity nodes, $E \subseteq V \times V$ is the set of relations, and $T$ is a set of relation-triples $(h, r, t)$ with $h, t \in V$. Given $q$, the goal is to return an answer $a$ that maximises faithfulness with respect to $\mathcal{D}$ while supporting an explicit reasoning chain over $G$. We formalise the retrieval step as the problem of selecting a subgraph $G_q \subseteq G$ that jointly (i) maximises a query-conditioned node-influence score and (ii) minimises a query-conditioned edge-cost, subject to a budget constraint on $|G_q|$.

### 5.2.2 Proposed Approach

We propose a three-step framework. Step 1 constructs a high-precision KG from $\mathcal{D}$; Step 2 retrieves an influence-aware reasoning subgraph for each query; Step 3 combines graph retrieval with conventional sparse and dense retrieval to produce the final context fed to the LLM. The pipeline is sketched in Figure 5.2.

#### Robust Knowledge Graph Construction

To mitigate Problem 1, we plan to decouple *entity extraction* from *relation extraction*. For entity extraction, we will rely on a statistical TF–IDF pipeline rather than on the LLM. Concretely, we will (i) tokenise each document and lemmatise candidate noun phrases; (ii) compute the TF–IDF weight of every candidate against the global corpus; and (iii) retain candidates whose weight exceeds a corpus-calibrated threshold, after applying linguistic filters that remove stopwords, dates, and numerical literals. The resulting candidate set

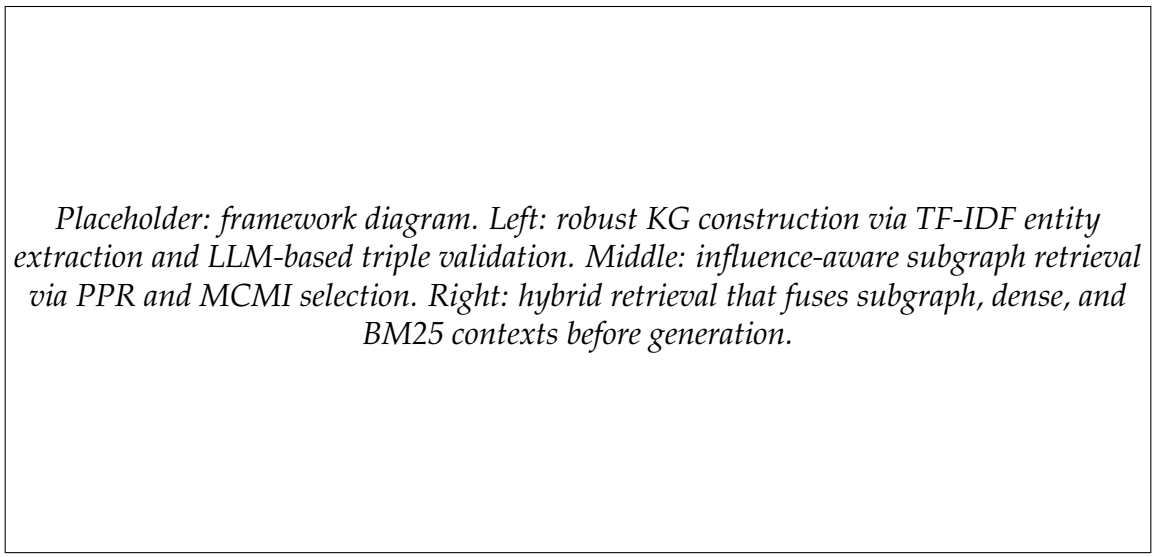



Figure 5.2: Proposed three-step framework for advanced graph-based RAG.

serves as the node set $V$ of the KG. Because TF–IDF is fully deterministic and computed only from observed text, this stage cannot hallucinate new mentions.

For relation extraction, we will retain an LLM but restrict its role to *validating* candidate triples. Given a passage $p$ and two co-occurring entities $u, v \in V$, we will prompt the LLM to decide whether $p$ asserts a relation between $u$ and $v$ and, if so, to label the relation $r$. The triple $(u, r, v)$ is added to $T$ only if the LLM returns a syntactically well-formed relation supported by a verbatim span of $p$. Because the LLM is confined to a classification-style task with constrained outputs, the surface for hallucination is much smaller than in free-form extraction.

We will further enrich each edge $e = (u, v) \in E$ with the set of supporting triples $T_e \subseteq T$ and the concatenated text of their evidence spans; these will be required by the influence-aware retrieval step.

**Influence-Aware Graph Retrieval**

Given a query $q$, we will first map $q$ to a set of *seed triples* $T_q \subseteq T$ by combining dense semantic similarity between $q$ and the textual rendering of each triple with a coarse keyword filter. The seed triples induce a set of seed nodes $V_q \subseteq V$.

**Node influence via Personalised PageRank.** We will compute Personalised PageRank (PPR) [25, 3] with the seed set $V_q$ as the personalisation distribution. The PPR vector $\boldsymbol{\pi}_q$ assigns each node $v \in V$ an influence score $\pi_q(v) \in [0, 1]$ that reflects its structural proximity to the query topic. PPR is particularly attractive here because efficient local algorithms allow $\boldsymbol{\pi}_q$ to be approximated in time sublinear in $|V|$, which is essential for sub-second online retrieval [25].

**Edge cost via query–edge semantic similarity.** We will assign each edge $e \in E$ a cost

$$c_q(e) \;=\; 1 - \mathrm{sim}\big(q,\; \mathrm{repr}(T_e)\big), \tag{5.1}$$

where $\mathrm{repr}(T_e)$ is the concatenated textual rendering of the triples on $e$ and $\mathrm{sim}(\cdot,\cdot)$ is a dense sentence-encoder similarity. Intuitively, edges whose supporting evidence is on-topic for $q$ have low cost.

**Minimum-cost maximum-influence subgraph.** The central algorithmic novelty of this direction is the *minimum-cost maximum-influence* (MCMI) subgraph problem. Given a budget $B$, we seek a connected subgraph $G_q^* = (V_q^*, E_q^*)$ with $|E_q^*| \leqslant B$ that maximises the bi-criterion objective

$$\mathcal{J}(G_q^*) \;=\; \sum_{v \in V_q^*} \pi_q(v) \;-\; \lambda \sum_{e \in E_q^*} c_q(e), \tag{5.2}$$

where $\lambda > 0$ trades influence against cost. Unlike Steiner-tree or shortest-path formulations, $G_q^*$ is allowed to contain cycles, so multiple disjoint reasoning chains that share intermediate entities are not artificially broken. The MCMI problem generalises the prize-collecting Steiner tree problem to arbitrary subgraphs and is therefore NP-hard.

We will design a greedy approximation that grows $G_q^*$ from the seed nodes $V_q$ by, at each step, adding the edge that maximises the marginal gain in $\mathcal{J}$ per unit cost, until the budget $B$ is exhausted. To support cycle inclusion, we will additionally allow non-tree edges whose marginal gain exceeds a topology-aware threshold. We will analyse the approximation ratio of this greedy procedure under monotone submodular relaxations of the node-influence component and report empirical optimality gaps on small instances solved by integer programming.

**Hybrid Retrieval Integration**

To address Problem 3, the retrieved subgraph $G_q^*$ will be linearised into a structured prompt fragment that names each entity, its role, and the supporting evidence span. The fragment will then be concatenated with two auxiliary contexts: (i) the top-$k_d$ passages returned by a dense retriever trained on the same corpus, and (ii) the top-$k_s$ passages returned by BM25 [30]. A lightweight re-ranker will reorder the merged context to remove redundancy before it is consumed by the LLM. By design, the dense and sparse channels compensate for evidence that did not reach the KG, while the graph channel contributes the explicit relational scaffolding that pure retrieval cannot provide.

### 5.2.3 Planned Experiments

**Datasets.** We will evaluate on three families of benchmarks: multi-hop question answering on HotpotQA [42] and the MuSiQue multi-hop dataset [35]; single-hop open-domain QA on Natural Questions [23]; and long-form summarisation on the GraphRAG community-summarisation benchmark [11]. To probe robustness, we will additionally construct a domain-shifted variant by sourcing documents from biomedical and legal corpora.

**Baselines.** The framework will be compared against (i) NaiveRAG, a top-k dense retriever followed by an LLM reader; (ii) BM25-only retrieval; (iii) GraphRAG [11]; (iv) LightRAG [14]; and (v) an ablated version of our pipeline that uses LLM-based entity extraction. The same backbone LLM (planned: GPT-4 [26] and an open-source 8B model) will be used across systems to isolate the effect of retrieval.

**Metrics.** We will report exact-match and F1 for short-form QA, faithfulness and citation-precision for long-form generation, and answer completeness assessed by an LLM-as-a-judge protocol calibrated against human annotations on a held-out subset.

**Ablation studies.** We plan to ablate (i) the entity-extraction component (TF–IDF vs. LLM); (ii) the retrieval objective (PPR + MCMI vs. shortest path vs. community summari-

sation); (iii) the hybrid integration (graph-only vs. graph+dense vs. graph+dense+BM25); and (iv) the budget $B$ and trade-off coefficient $\lambda$ of Equation (5.2). We will additionally measure end-to-end latency and indexing cost to characterise the practical operating points of the framework.

### 5.2.4 Expected Contributions

We anticipate four contributions from this direction. First, a robust KG construction protocol that decouples deterministic entity extraction from constrained relation validation. Second, a novel MCMI formulation of query-conditioned subgraph retrieval that admits cycles and unifies neighbourhood- and walk-based reasoning. Third, a hybrid retrieval architecture that demonstrates how structural and lexical channels can be combined without mutual interference. Fourth, an open implementation and benchmark suite that will allow the community to reproduce and extend the comparison against GraphRAG [11] and LightRAG [14].

## 5.3 Semantic Subgraph Matching and Graph-Aware Caching

### 5.3.1 Motivation and Problem Definition

The framework proposed in Section 5.2 exposes two data-management problems that are general to any graph-augmented LLM system at scale.

**Problem 1: Inflexible subgraph matching.** Classical subgraph isomorphism algorithms such as VF2 [7] test whether a query graph $G_q$ is topologically embedded in a data graph $G_d$. They treat node and edge labels as opaque symbols. In a knowledge-graph setting, however, two graphs that are not isomorphic may still encode the same information. For instance, a query graph with a direct edge $A \rightarrow C$ ought to match a data graph that mediates the same relation through an intermediate node, $A \rightarrow B \rightarrow C$, when the path $A \rightarrow B \rightarrow C$ is semantically equivalent to $A \rightarrow C$. Strict topological matching misses such equivalences and therefore degrades both recall in graph-RAG retrieval and precision in graph-based data integration.

**Problem 2: Event-agnostic KV cache recomputation.** LLM inference relies on a key–value (KV) cache to amortise self-attention computation across tokens. State-of-the-art KV-cache eviction policies score tokens by recency or attention magnitude and are blind to the structural role of the underlying content. When the prompt is constructed from a retrieved subgraph, however, tokens that correspond to high-influence nodes carry far more downstream signal than tokens drawn from peripheral evidence. An event-agnostic eviction policy may discard precisely the tokens that anchor the reasoning chain, forcing expensive recomputation when subsequent queries revisit the same subregion of the graph.

**Problem statement.** Let $G_d = (V_d, E_d)$ be a data knowledge graph and $G_q = (V_q, E_q)$ be a query graph with node and edge attributes. The *path-aware semantic subgraph matching* problem asks for the set of mappings $\phi : V_q \hookrightarrow V_d$ that are consistent with $G_d$ either topologically or via a path-level semantic equivalence relation $\sim_P$ defined over paths of bounded length. The *graph-aware KV cache* problem asks for an eviction policy that, given the set of subgraphs likely to be retrieved by an online query stream, retains in cache the tokens whose exclusion would induce the highest expected recomputation cost.

### 5.3.2 Proposed Approach

#### Path-Aware Semantic Subgraph Matching

We plan to extend the VF2 enumeration backbone [7] with two modifications. First, we will replace the discrete label-equality test by a *semantic compatibility* test that returns true when the dense embeddings of the candidate label pair lie within a learned similarity threshold. Second, and more importantly, we will introduce *path substitution rules*: when the matcher fails to match an edge $(u, v) \in E_q$ directly in $G_d$, it will attempt to match the edge against any path $u \to w_1 \to \cdots \to w_k \to v$ in $G_d$ whose linearised semantics is judged equivalent to the query edge under a path-encoder model.

**Path encoder.** We will train a path encoder $\Phi$ that maps a sequence of triples to a fixed-dimensional vector. Equivalence between a query edge $(u, r, v)$ and a data path $\pi$ will

be decided by the cosine similarity between $\Phi(\{(u, r, v)\})$ and $\Phi(\pi)$. We will mine training pairs from KG completion corpora and from the documents underlying the corpora described in Section 5.2.3.

**Candidate pruning.** Because admitting path substitutions enlarges the search space, we will design pruning rules based on path length, edge cost (reusing the formulation of Section 5.2.2), and upper bounds on the path-encoder similarity. We anticipate that these rules will preserve the asymptotic behaviour of VF2 while exposing semantic matches that are unreachable to purely topological matchers.

#### Graph-Aware KV Cache Mechanism

To address Problem 2 we will design a cache controller that scores each contiguous block of cached tokens by an *expected reuse benefit* combining (i) the average node-influence score (Equation (5.2)) of the underlying KG nodes; (ii) the empirical frequency with which the corresponding nodes appear in recently retrieved subgraphs; and (iii) the attention magnitude observed during previous generations. Eviction will prioritise blocks with low expected reuse benefit. The controller will be agnostic to the underlying LLM and will be implemented as a lightweight layer on top of standard transformer inference engines.

We will additionally study a *predictive prefetch* variant in which the controller, after observing a query that retrieves subgraph $G_q^*$, proactively re-warms the KV cache for high-influence neighbours of $G_q^*$ that are predicted to participate in follow-up queries within the same session. The prefetch policy will be cast as a contextual bandit driven by features of the query stream.

### 5.3.3 Planned Experiments

**Datasets.** For subgraph matching, we will use knowledge-graph QA benchmarks such as MetaQA [44] and ComplexWebQuestions [34], together with the document-level KGs constructed in Section 5.2.3. To explore multi-modality, we will additionally evaluate on a vision-language KG benchmark such as WebQA [5]. For KV caching, we will reuse the

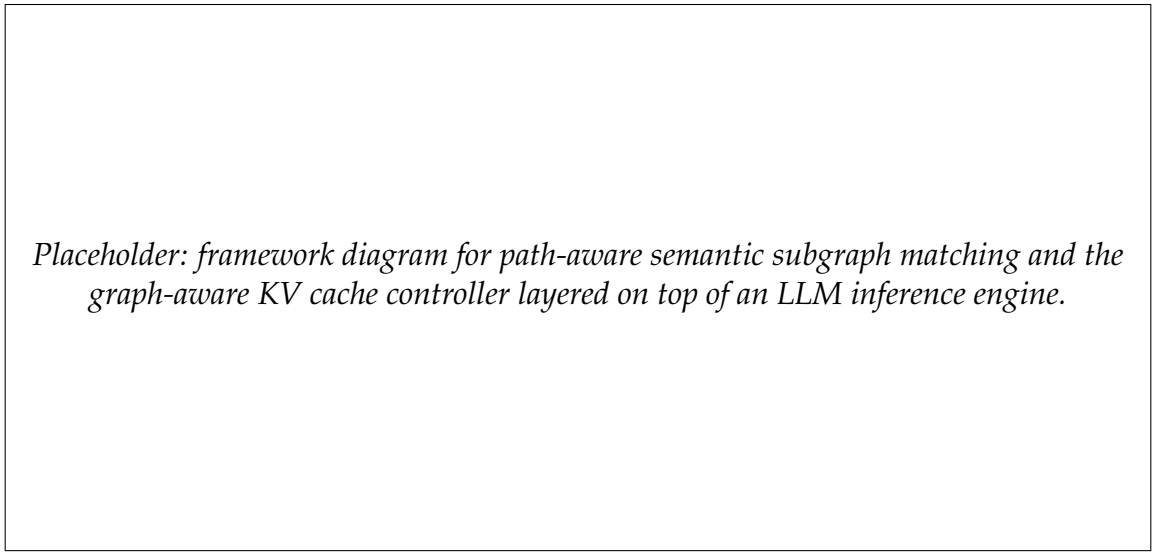



Figure 5.3: Proposed framework for semantic subgraph matching and graph-aware KV caching.

long-context QA traces produced by the framework of Section 5.2 and synthesise session-level workloads that exhibit temporal locality.

**Baselines.** The semantic matcher will be compared against VF2 [7], an approximate subgraph-matching baseline based on embedding similarity, and a recent GNN-based matcher. The KV cache mechanism will be compared against the default rolling-window cache, an attention-based eviction policy, and an oracle that has access to future queries.

**Metrics.** For matching, we will report matching accuracy (precision/recall against gold semantic matches), end-to-end retrieval latency, and downstream answer quality when the matcher feeds a graph-RAG reader. For caching, we will report cache hit rate, prefill latency, time to first token, and generation quality. A scalability analysis will measure each metric as a function of graph size, query graph size, and concurrent session count.

**Ablation studies.** We will ablate (i) the path-substitution mechanism (on/off), (ii) the path-encoder backbone, (iii) the contribution of each term of the expected reuse benefit, and (iv) the prefetch policy.

### 5.3.4 Expected Contributions

We expect this direction to deliver three contributions: a path-aware semantic subgraph matching algorithm that bridges topological and semantic equivalence; a graph-aware KV cache controller that exploits structural importance to reduce inference cost; and an open benchmark suite that quantifies the system-level benefits of structural awareness for graph-augmented LLM serving.

## 5.4 Research Timeline

The two proposed directions will be executed in parallel, with the data-management contributions of Section 5.3 consuming the workloads and indices produced by Section 5.2. Table 5.1 summarises the planned milestones from 2026 through 2028, culminating in dissertation writing and defence.

Successful execution of this timeline will deliver a coherent body of work that advances structural augmented reasoning from the task-specific contributions of KGLink [40] and GORAG [39] to a general-purpose, system-aware framework for graph-augmented language model reasoning. The two proposed directions are individually publishable and collectively constitute the technical core of the doctoral thesis.

Table 5.1: Planned research timeline (2026–2028).

| **Period** | **Direction** | **Milestone** |
|---|---|---|
| 2026 Q2–Q3 | Direction 1 | Implement robust KG construction (TF–IDF + LLM triple validation); release indexed corpora for HotpotQA and Natural Questions. |
| 2026 Q3–Q4 | Direction 1 | Develop PPR-based influence scoring and greedy MCMI subgraph retrieval; preliminary experiments against GraphRAG and LightRAG. |
| 2026 Q4–2027 Q1 | Direction 1 | Integrate hybrid graph+dense+BM25 retrieval; full benchmark sweep; submit first paper on advanced graph-based RAG. |
| 2027 Q1–Q2 | Direction 2 | Design and prototype path-aware semantic subgraph matcher; train path encoder on KG completion data. |
| 2027 Q2–Q3 | Direction 2 | Benchmark matcher against VF2 and approximate baselines on KGQA and document-level KGs; integrate matcher into the Direction 1 retrieval pipeline. |
| 2027 Q3–Q4 | Direction 2 | Implement graph-aware KV cache controller and predictive prefetch; measure inference-time benefits on long-context QA traces. |
| 2027 Q4–2028 Q1 | Direction 2 | Scalability study on multi-modal benchmarks; submit second paper on semantic subgraph matching and graph-aware caching. |
| 2028 Q1–Q2 | Cross-cutting | Unified system release combining the two directions; ablations on end-to-end latency, faithfulness, and cost. |
| 2028 Q2–Q3 | Dissertation | Thesis writing, internal defence, and final submission. |
| 2028 Q3–Q4 | Dissertation | Public defence and dissemination. |

# CHAPTER 6

# CONCLUSION AND EXPECTED CONTRIBUTIONS

This chapter concludes the thesis proposal by summarizing the completed work, articulating the expected contributions across theoretical, methodological, and practical dimensions, and discussing potential risks together with mitigation strategies. We also outline alternative plans to ensure the robustness of the overall research agenda.

## 6.1 Summary of Completed Work

The central thesis of this proposal is that explicit structured representations—such as knowledge graphs and dynamically constructed weighted graphs—can substantially enhance the reasoning capabilities of language models when integrated into their processing pipelines. The two completed works presented in this proposal provide strong empirical evidence for this claim.

In Chapter 3, we introduced **KGLink**, a hybrid framework that combines knowledge graph representations with pre-trained language models for column type annotation [40]. KGLink addresses two critical challenges that had limited prior approaches: the type granularity problem, where fine-grained types are difficult to disambiguate from coarse labels alone, and the missing context problem, where tabular data lacks sufficient surrounding information for accurate type inference. By leveraging structured knowledge from external knowledge graphs as an augmentation to the language model's input, KGLink demonstrates that explicit relational structure can resolve ambiguities that purely text-based approaches cannot. The success of KGLink validates our core hypothesis in the domain of structured data understanding.

In Chapter 4, we presented **GORAG**, a graph-based online retrieval-augmented generation framework for dynamic few-shot text classification on social media [39]. GORAG constructs adaptive weighted graphs that capture semantic and contextual relationships

among candidate examples, and employs a minimum-cost spanning tree retrieval mechanism to select optimal demonstration sets for each input instance. Unlike prior approaches that rely on static retrieval strategies or fixed thresholds, GORAG adapts its retrieval topology to the specific characteristics of each query. The publication of GORAG at WWW 2026 confirms the practical value and novelty of graph-structural augmentation for retrieval-based reasoning in dynamic, real-world settings.

Together, these two works establish the feasibility and effectiveness of structural augmented reasoning across distinct application domains—tabular data integration and social media content classification—laying a solid foundation for the more ambitious future directions proposed in Chapter 5.

## 6.2 Expected Contributions

### 6.2.1 Theoretical Contributions

This thesis is expected to make several theoretical contributions to the understanding of how structured representations interact with language model reasoning.

First, we formalize the notion of *structural augmented reasoning* as a unified paradigm that encompasses diverse instantiations, from knowledge graph augmentation (KGLink) to dynamic graph construction (GORAG) to influence-aware subgraph generation (proposed work). This formalization provides a conceptual framework for reasoning about when and why structural augmentation improves language model performance, and identifies the key design dimensions—graph construction policy, retrieval mechanism, and integration strategy—that determine system effectiveness.

Second, the proposed work on MCMI (Minimum-Cost Maximum-Influence) subgraph generation in Section 5.2 of Chapter 5 includes a theoretical analysis of the computational complexity of the problem. We establish the NP-hardness of optimal subgraph selection under joint cost and influence constraints and provide approximation guarantees for the proposed greedy algorithm. This theoretical grounding ensures that our practical solutions are anchored in rigorous computational foundations rather than purely empirical observations.

Third, the path-aware semantic subgraph matching framework proposed in Section 5.3 of Chapter 5 contributes a formal framework for flexible query-document graph alignment. By defining path-level semantic similarity measures and establishing their relationship to downstream reasoning quality, we provide theoretical tools for understanding the granularity at which structural matching should operate.

### 6.2.2 Methodological Contributions

The methodological contributions of this thesis span both the completed and proposed works.

**KGLink** represents the first hybrid KG+PLM method that simultaneously addresses both the type granularity and missing context problems for column type annotation. Its multi-source integration strategy—combining entity-level knowledge graph information with column-level statistical features—establishes a methodological template for augmenting language models with structured external knowledge in data integration tasks.

**GORAG** introduces the first adaptive graph-based RAG framework with threshold-free, input-specific retrieval for text classification. The key methodological innovation lies in the online graph construction and minimum-cost spanning tree retrieval, which together enable the system to automatically calibrate the number and diversity of retrieved examples without requiring task-specific hyperparameter tuning.

The proposed **influence-aware subgraph generation** for advanced graph-based RAG contributes a novel methodology for constructing retrieval subgraphs that explicitly account for the influence relationships between knowledge elements. The Personalized PageRank-based scoring mechanism combined with MCMI optimization provides a principled approach to selecting subgraphs that maximize reasoning support while minimizing hallucination-inducing noise.

The proposed **graph-aware KV cache** integrates topological importance scores into the key-value cache management of transformer models, representing a methodological contribution at the intersection of graph algorithms and efficient inference. This approach enables long-context processing that respects the structural relationships among cached tokens, rather than relying solely on recency or attention-based eviction policies.

### 6.2.3 Practical Contributions

Beyond theoretical and methodological advances, this thesis makes concrete practical contributions to the research community and potential industrial applications.

On the implementation front, the GORAG codebase has been released as open-source software (available at github.com/Wyb0627/GORAG), enabling reproducibility and facilitating follow-up research. We intend to release implementations of the proposed methods upon completion as well.

The techniques developed in this thesis are directly applicable to real-world scenarios of significant practical importance. KGLink addresses data integration challenges faced by enterprises managing heterogeneous data sources. GORAG enables effective content moderation and classification on social media platforms where label distributions evolve rapidly. The proposed advanced RAG methods target knowledge-intensive question answering, a capability increasingly demanded in enterprise AI systems, customer support, and scientific research assistance.

Furthermore, all proposed methods are designed with scalability as a first-class concern. The greedy approximation algorithms for MCMI subgraph generation, the lightweight scoring functions for graph-aware caching, and the TF-IDF-based graph construction approach are all intended to operate efficiently at production scale, making them viable candidates for deployment in real-world systems.

## 6.3 Potential Risks and Mitigation Strategies

### 6.3.1 Technical Risks

The proposed research involves several technical challenges that could potentially impede progress.

The MCMI subgraph generation problem is NP-hard, which means that exact optimal solutions are computationally intractable for large knowledge graphs. We mitigate this risk through a greedy approximation algorithm that exploits the submodularity properties of influence functions to provide provable approximation bounds. Preliminary analysis

suggests that the greedy approach achieves near-optimal solutions in practice while maintaining linear-time complexity with respect to the number of candidate nodes. Should the greedy approach prove insufficient for specific graph topologies, we have identified additional pruning strategies based on structural properties (e.g., connectivity, degree distribution) that can further reduce the search space.

Graph construction quality is a second technical concern, particularly given that entity extraction from noisy or ambiguous text can introduce errors that propagate through the graph structure. We address this risk by adopting a TF-IDF-based statistical approach for initial graph construction rather than relying on potentially hallucination-prone LLM-based extraction. This statistical grounding provides a more reliable foundation, and our influence-scoring mechanism is designed to be robust to moderate levels of noise in the underlying graph structure.

The graph-aware KV cache mechanism introduces additional computation during inference, which could potentially offset the efficiency gains from reduced context length. We mitigate this risk by designing lightweight scoring functions that operate in constant or logarithmic time per token, amortizing the graph-awareness overhead across multiple decoding steps. Early design analysis indicates that the overhead can be kept below 5% of total inference time while achieving meaningful improvements in long-context reasoning quality.

### 6.3.2 Data and Evaluation Risks

Research in retrieval-augmented generation faces several data and evaluation challenges that we have proactively considered.

The availability of suitable evaluation datasets is a potential concern, particularly for novel task formulations. We mitigate this by leveraging well-established public benchmarks including HotpotQA, Natural Questions, MuSiQue, and 2WikiMultiHopQA for knowledge-intensive question answering evaluation. These datasets provide diverse reasoning challenges and enable fair comparison with existing methods. For domain-specific evaluation, we plan to construct targeted test sets from publicly available knowledge bases.

The lack of unified evaluation standards for RAG systems presents a challenge for demonstrating the superiority of proposed methods. We address this by adopting a multi-dimensional evaluation protocol that assesses accuracy (correctness of final answers), faithfulness (groundedness in retrieved evidence), completeness (coverage of relevant information), and efficiency (computational cost). This comprehensive evaluation framework ensures that improvements are robust across multiple quality dimensions rather than optimizing for a single metric.

For GORAG and its extensions to dynamic settings, simulating realistic label evolution is inherently difficult. We mitigate this by using real-world temporal datasets from social platforms where natural distribution shift is observable, rather than relying on synthetic perturbations that may not reflect actual deployment conditions.

### 6.3.3 Timeline Risks

The proposed research agenda is ambitious, encompassing two substantial future projects each targeting venue-quality publications. We acknowledge the timeline risk inherent in this scope and have designed several mitigations.

Each proposed project is structured to be independently publishable, meaning that partial completion of the overall agenda still yields meaningful research contributions. The advanced graph-based RAG project (Section 5.2) and the semantic subgraph matching with graph-aware caching project (Section 5.3) can proceed in parallel to some degree, as they address complementary aspects of the structural augmented reasoning paradigm.

To further mitigate timeline risks, we plan to begin prototype development concurrently with theoretical analysis, rather than following a strictly sequential theory-then-implementation approach. This parallel development strategy allows early identification of practical challenges that might require theoretical reformulation, reducing the risk of late-stage surprises.

## 6.4 Alternative Plans

We have developed contingency plans for scenarios where primary approaches encounter fundamental obstacles.

If the MCMI optimization approach faces scalability issues that cannot be adequately addressed through greedy approximation, we will simplify the retrieval structure to tree-based representations with pruning heuristics. Tree-structured retrieval sacrifices some expressiveness compared to general subgraph retrieval but offers guaranteed polynomial-time complexity and has proven effective in related work on hierarchical document retrieval.

If semantic subgraph matching proves computationally prohibitive for real-time applications, we will pivot to an approximate matching strategy based on graph embedding similarity. By encoding subgraph structures into dense vector representations, we can leverage efficient nearest-neighbor search infrastructure to achieve sub-linear retrieval time at the cost of some matching precision.

If the graph-aware KV cache demonstrates limited empirical gains over standard cache management policies, we will redirect this line of investigation toward selective graph summarization for context compression. Rather than modifying the cache mechanism itself, this alternative approach would use graph structure to identify and compress redundant information before it enters the context window, achieving similar goals through a different technical path.

Finally, if the primary proposed directions encounter fundamental theoretical or empirical obstacles, we have identified cross-language and multilingual RAG as a viable fallback research direction. The structural augmented reasoning framework naturally extends to multilingual settings where knowledge graphs can serve as language-agnostic bridges, and this direction aligns well with the expertise developed through the completed works.

## 6.5 Concluding Remarks

This thesis proposal presents a coherent research program organized around a unifying vision: that explicit structured representations serve as a critical bridge between raw knowledge sources and the reasoning capabilities of language models. By making relational structure explicit and algorithmically accessible, we enable language models to perform more reliable, interpretable, and efficient reasoning across diverse tasks.

The contributions of this thesis span a deliberate progression from specific applications to general infrastructure. KGLink (Chapter 3) demonstrates structural augmentation for a focused data integration task. GORAG (Chapter 4) generalizes the approach to dynamic retrieval scenarios in social media understanding. The proposed future works (Chapter 5) further extend the paradigm to general-purpose knowledge-intensive reasoning and efficient long-context processing, addressing fundamental challenges in the rapidly evolving landscape of large language model systems.

We anticipate that the completed and proposed works will collectively advance both academic understanding of structure-aware language model reasoning and the practical capabilities of AI systems deployed in real-world applications. The structural augmented reasoning paradigm offers a principled path toward language models that are not only powerful but also grounded, efficient, and trustworthy—qualities that are essential for the responsible deployment of AI technology in knowledge-critical domains.

# REFERENCES


[1] Nora Abdelmageed and Sirko Schindler. Jentab: Matching tabular data to knowledge graphs. In *CEUR Workshop Proceedings (SemTab@ISWC)*, 2020.

[2] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. Dbpedia: A nucleus for a web of open data. In *The Semantic Web*, pages 722–735. Springer, 2007.

[3] Bahman Bahmani, Kaushik Chakrabarti, and Dong Xin. Fast personalized pagerank on mapreduce. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, pages 973–984, 2011.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[5] Yingshan Chang, Mridu Narang, Hisami Suzuki, Guihong Cao, Jianfeng Gao, and Yonatan Bisk. Webqa: Multihop and multimodal qa. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16495–16504, 2022.

[6] Jiaoyan Chen, Ernesto Jiménez-Ruiz, Ian Horrocks, and Charles Sutton. Colnet: Embedding the semantics of web tables for column type prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 29–36, 2019.

[7] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. A (sub)graph isomorphism algorithm for matching large graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(10):1367–1373, 2004.

[8] Thomas Davidson, Dana Warmsley, Michael Macy, and Ingmar Weber. Automated hate speech detection and the problem of offensive language. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 11, pages 512–515, 2017.

[9] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. Turl: Table understanding through representation learning. *Proceedings of the VLDB Endowment*, 14(3):307–319, 2022.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

[11] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[12] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1126–1135, 2017.

[13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. In *arXiv preprint arXiv:2312.10997*, 2023.

[14] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

[15] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. In *ACM Computing Surveys*, volume 54, pages 1–37, 2021.

[16] Maarten Houbraken, Sofie Demeyer, Tom Michoel, Pieter Audenaert, Didier Colle, and Mario Pickavet. Index-based subgraph matching algorithm (ismags): An existing open-source data mining tool. *PLoS ONE*, 9(4):e97896, 2014.

[17] Madelon Hulsebos, Kevin Hu, Michiel Bakker, Emanuel Zgraggen, Arvind Satyanarayan, Tim Kraska, Çağatay Demiralp, and César Hidalgo. Sherlock: A deep learn-

ing approach to semantic data type detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1500–1508, 2019.

[18] Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. Tabbie: Pretrained representations of tabular data. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3446–3456, 2021.

[19] Shaoxiong Ji, Celina Ping Yu, Sai-fu Fung, Shirui Pan, and Guodong Long. Suicidal ideation detection: A review of machine learning methods. *IEEE Transactions on Computational Social Systems*, 8(1):214–226, 2021.

[20] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[21] Ernesto Jiménez-Ruiz, Oktie Hassanzadeh, Vasilis Efthymiou, Jiaoyan Chen, Kavitha Srinivas, and Vincenzo Cutrona. Semtab 2020: Semantic web challenge on tabular data to knowledge graph matching. In *CEUR Workshop Proceedings*, volume 2775, 2020.

[22] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.

[23] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, and Kenton Lee. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9403–9414, 2020.

[25] Peter Lofgren, Siddhartha Banerjee, and Ashish Goel. Personalized pagerank estimation and search: A bidirectional approach. In *Proceedings of the Ninth ACM International Conference on Web Search and Data Mining*, pages 163–172, 2016.

[26] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[27] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[28] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.

[29] Dominique Ritze, Oliver Lehmberg, and Christian Bizer. Matching html tables to dbpedia. In *Proceedings of the 5th International Conference on Web Intelligence, Mining and Semantics*, pages 1–6, 2015.

[30] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[31] Kai Shu, Amy Sliva, Suhang Wang, Jiliang Tang, and Huan Liu. Fake news detection on social media: A data mining perspective. *ACM SIGKDD Explorations Newsletter*, 19(1):22–36, 2017.

[32] Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[33] Yoshihiko Suhara, Jinfeng Li, Yuliang Li, Dan Zhang, Çağatay Demiralp, Chen Chen, and Wang-Chiew Tan. Annotating columns with pre-trained language models. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 1493–1508, 2022.

[34] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 641–651, 2018.

[35] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[37] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Koray Kavukcuoglu, and Daan Wierstra. Matching networks for one shot learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[38] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.

[39] Yubo Wang, Haoyang Li, Fei Teng, and Lei Chen. Gorag: Graph-based online retrieval augmented generation for dynamic few-shot social media text classification. In *Proceedings of the ACM Web Conference 2026*, 2026.

[40] Yubo Wang, Hao Xin, and Lei Chen. Kglink: A column type annotation method that combines knowledge graph and pre-trained language model. *arXiv preprint arXiv:2406.00318*, 2024.

[41] Zhiruo Wang, Haoyu Dong, Ran Jia, Jia Li, Zhiyi Fu, Shi Han, and Dongmei Zhang. Tuta: Tree-based transformers for generally structured table pre-training. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 1780–1790, 2021.

[42] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

[43] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. Tabert: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8413–8426, 2020.

[44] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J. Smola, and Le Song. Variational reasoning for question answering with knowledge graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[45] Ziqi Zhang. Effective and efficient semantic table interpretation using tableminer+. In *Journal of Web Semantics*, volume 44, pages 33–53, 2017.